SCHOOL OF ARCHITECTURE, COMPUTING AND ENGINEERING

Department of Engineering and Computing

Illuminating the Black Box: An Explainable AI Framework for Brain Tumour Segmentation Using Volumetric Data Interpretation in 3D CNNs

Yogi Amitkumar Patel

2536809

A report submitted in part fulfilment of the degree of
BSc (Hons) in *Data Science and Artificial Intelligence*

Supervisor: Maimoona Sarif

CN6000

16 March 2026

Abstract

Despite achieving state-of-the-art performance in neuro-oncological imaging, deep learning models remain clinically underutilised due to their opacity - a critical barrier when automated segmentation informs high-stakes decisions such as brain tumour resection planning. This dissertation addresses the fundamental tension between predictive accuracy and clinical trustworthiness by developing an integrated framework for explainable and uncertainty-aware volumetric brain tumour segmentation. A 3D SegResNet architecture was trained on the BraTS 2023 dataset (n = 1,251 glioma patients) to segment three clinically salient sub-regions - Whole Tumour (WT), Tumour Core (TC), and Enhancing Tumour (ET) - from multimodal MRI, achieving Dice scores of 0.923, 0.891, and 0.873 respectively, with an ET boundary precision of 3.66 mm HD95, surpassing BraTS 2023 challenge medians across all sub-regions.

The principal contribution lies in a systematic, multi-modal explainability framework that renders the model's decision-making process auditable through six complementary post-hoc attribution methods: Grad-CAM, Guided Backpropagation, Guided Grad-CAM, Input × Gradient, Occlusion Sensitivity, and Monte Carlo Dropout. Saliency fidelity was quantitatively evaluated via Pointing Game accuracy, Saliency Coverage, and Saliency IoU, ensuring methodological rigour beyond qualitative impression. A key empirical finding, derived from analysis across 25 held-out patients, is that saliency and uncertainty constitute statistically independent signals (Pearson $r \approx 0$), establishing their non-redundant complementarity for clinical decision support. To overcome the spatial information loss inherent in conventional 2D displays, 3D saliency and uncertainty volumes were streamed into an immersive Virtual Reality environment, rendered via GPU-accelerated ray-casting and delivered wirelessly to a Meta Quest 3 headset through 3D Slicer, enabling stereoscopic, six-degrees-of-freedom navigation within anatomical context.

Current limitations include the absence of formal clinician user studies, reliance on a single data split without cross-validation, and partial implementation of XAI-guided model pruning. Future work will prioritise within-subjects radiologist evaluation measuring diagnostic accuracy and NASA-TLX cognitive load across flat-panel versus VR workflows, multi-institutional external validation, and completion of the feedback-driven pruning loop. Ultimately, this work demonstrates that the "black box" nature of deep learning is not an inherent epistemological limit but a tractable engineering problem; through rigorous volumetric attribution, principled uncertainty quantification, and immersive reproducible visualisation, we establish a pathway toward clinically trustworthy neuro-oncological AI.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Maimoona Sarif, for her invaluable guidance, expertise, and continuous support throughout the development of this dissertation. Her insightful feedback and encouragement were instrumental in shaping both the direction and scope of this research.

I am deeply grateful to my close friend, Jayrup Nakawala, whose technical assistance was indispensable to this work. His support in configuring computational infrastructure and facilitating access to remote server resources for model training enabled the full-scale experiments that underpin the results presented herein.

I also extend my heartfelt thanks to Maged Abdelmonem for generously providing access to a VR headset, which was essential to the immersive visualisation component of this framework and directly contributed to achieving the project's objectives. Finally, I am profoundly thankful to my family and friends for their unwavering patience, understanding, and motivation during the demanding months of this final year project.

Contents

List of Figures

List of Tables

Chapter 1: Introduction

Every year, approximately 308,000 people worldwide receive a primary brain tumour diagnosis (Neri *et al.*, 2023). For the neurosurgeon holding the pre-operative scan, the critical question is never simply **whether** a tumour is present it is **where, exactly, does it end?** A margin error of a few millimetres in the wrong direction means either residual malignant tissue or permanent neurological deficit. This dissertation begins at that boundary.

1.1 Clinical Motivation

Magnetic Resonance Imaging (MRI) has become the gold standard for neuro-oncological assessment, offering the sub-millimetre volumetric resolution required to characterise the three clinically distinct tumour subregions defined by the Brain Tumour Segmentation (BraTS) challenge: the Whole Tumour (WT), encompassing all pathological tissue; the Tumour Core (TC), comprising the necrotic and actively growing regions; and the Enhancing Tumour (ET), the contrast-enhancing rim that directly governs radiotherapy dose planning (Menze *et al.*, 2015). Manually delineating these boundaries from multi-modal MRI across T1, T1ce, T2, and FLAIR sequences is a labour-intensive, subjective, and anatomically demanding task. Inter-rater variability in manual segmentation has been documented at up to 28% for ET (Neri *et al.*, 2023), the subregion of highest clinical consequence. The case for automated, precise volumetric segmentation is unambiguous.

1.2 The Black Box Paradox

Deep learning has answered the accuracy imperative. Modern 3D Convolutional Neural Networks (CNNs), trained on large-scale benchmarks such as BraTS, routinely achieve Dice scores above 0.90 for Whole Tumour performance that matches or exceeds expert radiologists on standardised test sets (Bhati, Neha and Amiruzzaman, 2024). Yet this technical success has precipitated a crisis of trust. As these models grow deeper and more accurate, their internal decision-making becomes progressively opaque: a clinician receives a segmentation mask with no insight into why specific voxels were classified as tumour tissue. In a medicolegal environment governed by the GDPR's "right to explanation" (Neri *et al.*, 2023), this opacity is not merely an academic inconvenience, it is a regulatory and ethical barrier to clinical deployment.

This is the **black box paradox** of medical AI: the more capable the model, the less accountable it becomes. A clinician cannot responsibly act on a prediction they cannot interrogate. Trust, in clinical practice, is not granted to accuracy figures; it is earned through transparency.

1.3 The Research Response

Explainable AI (XAI) has emerged as the discipline that confronts this paradox directly. Post-hoc attribution techniques including gradient-based methods such as Grad-CAM and Guided Backpropagation, decomposition-based methods such as Layer-wise Relevance Propagation, perturbation-based methods such as Occlusion Sensitivity, and stochastic uncertainty methods such as Monte Carlo Dropout each offer a distinct window into the model's spatial reasoning (Selvaraju *et al.*, 2017; Natekar, Kori and Krishnamurthi, 2020). However, a second barrier immediately presents itself: even when volumetric XAI saliency maps are successfully generated, they are routinely visualised as 2D slice overlays on flat-panel displays. This impoverished representation forces clinicians to mentally reconstruct a fundamentally three-dimensional attribution volume from sequential cross-sections reintroducing precisely the cognitive burden that automation was intended to eliminate (Khedir *et al.*, 2025).

The perceptual solution is immersive Virtual Reality (VR). By delivering stereoscopic depth cues and six degrees-of-freedom navigation, VR eliminates the dimensional compression of slice based workflows, allowing a clinician to inhabit the tumour space, rotate attribution volumes, and inspect the spatial relationship between model confidence and anatomical structure directly (Zeineldin *et al.*, 2022). Yet no existing framework combines high fidelity 3D segmentation, a comprehensive multi paradigm XAI evaluation suite, and a reproducible open source VR visualisation pipeline into a single, end-to-end clinical proof-of-concept.

1.4 Research Aim and Objectives

This dissertation develops and evaluates precisely such a framework. The central aim is:

"To develop an Explainable AI framework that transforms an opaque 3D deep learning segmentation model into a transparent, trustworthy, and immersively interpretable decision-support system for brain tumour neuro-oncology."

This aim is pursued through seven objectives: establishing voxel-wise XAI localisation mechanisms; analysing the accuracy-transparency trade-off; quantitatively and qualitatively evaluating 3D XAI visualisations in VR; conducting robustness testing on XAI-attributed regions; training SegResNet on BraTS 2023 and integrating six post-hoc XAI techniques; conducting a feasibility analysis of XAI-guided architectural pruning, designing a conceptual pipeline and identifying the technical barriers to implementation; and evaluating limitations with proposed future directions.

The black box is not an intrinsic property of deep learning. It is a design choice and this dissertation is the argument for choosing differently.

Chapter 2: Literature Review

2.1 Introduction

2.1.1 Clinical Background and the "Black Box" Challenge

In modern oncology, properly identifying and managing brain tumours continues to be a major issue. Due to its simplicity and excellent volumetric resolution, Magnetic Resonance Imaging (MRI) has become the highest standard for analysis (Menze *et al.*, 2015). However, there is a substantial barrier in the interpretation of this data. At the moment, the process of defining the boundaries of tumours is mostly done by hand. Subjectivity and labour intensity are introduced by this reliance on human interpretation, which may not be sustainable for scaling healthcare solutions (Iftikhar *et al.*, 2025). Automated technologies that can match or surpass human precision without the corresponding time restrictions are therefore desperately needed in medical applications.

The response to this clinical need has been the rapid adoption of Deep Learning (DL) technologies, particularly 3D Convolutional Neural Networks (CNNs) like the 3D U-Net. These models have demonstrated "superior performance" in handling volumetric data compared to traditional methods (Bhati, Neha and Amiruzzaman, 2024; Iftikhar *et al.*, 2025). However, the integration of these advanced models has introduced a new, critical issue: the **"black box"** problem.

While these models are highly accurate, their internal decision-making processes are opaque (Neri *et al.*, 2023). **This creates a paradox in medical AI**: as models become more complex and accurate, they often become less interpretable to the clinicians using them. In a high-stakes environment like neurosurgery, a lack of transparency creates a "trust gap" (Wen *et al.*, 2025). It is not enough for a model to simply output a segmentation mask; clinicians require the ability to verify why specific voxels were classified as tumour tissue. Therefore, the lack of interpretability is no longer just a technical issue, but a barrier to ethical and safe clinical deployment (Neri *et al.*, 2023).

2.1.2 Research Aims and Scope of Literature Review

To address this disconnect between model accuracy and clinical trust, this literature review aims to critically evaluate the intersection of 3D deep learning, Explainable AI (XAI), and immersive visualisation. While these fields are often studied in isolation, this review argues that a unified framework is necessary to fully illuminate the "black box" of 3D CNNs.

The review is structured to build this argument logically:

Section 2.2 establishes the technical capabilities of modern 3D CNN architectures for segmentation.

Section 2.3 critically analyses current XAI methods (such as Grad-CAM and LIME), evaluating their limitations when applied to complex volumetric data.

Section 2.4 explores the potential of immersive technologies (VR) to present these XAI outputs in a way that is intuitively understandable for clinicians.

Section 2.5 synthesises these findings to identify the specific research gap this project will address: the lack of an integrated, 3D-visualised explainability pipeline for brain tumour segmentation.

Section 2.6 explains the investigation's findings and outlines how this initiative will fill the identified gap.

2.2 3D Deep Learning for Medical Brain Tumour Segmentation

2.2.1 Evaluation of 3D CNNs for Brain Tumour Segmentation

Over the past decade, the field of medical image analysis has experienced a significant paradigm shift, moving from **"reasoning-based"** systems that relied on manual feature engineering to **"learning-based"** deep learning (DL) approaches (Neri *et al.*, 2023). In the past, radiomics was used for the automated analysis of brain tumours. To train classifiers like Support Vector Machines (SVMs), domain specialists manually retrieved quantitative features including texture, intensity, and shape. However, this method was labour-intensive, prone to observer bias, and frequently failed to capture the intricate, non-linear spatial relationships present in volumetric data (Bhati, Neha and Amiruzzaman, 2024).

The introduction of Convolutional Neural Networks (CNNs) brought an end to this manual paradigm. CNNs take an end-to-end approach to extract features automatically by utilising raw pixel data in contrast to classical machine learning. Initial implementation in neuro-oncology treated Magnetic Resonance Imaging (MRI) as a series of 2D slices. While this approach allowed taking pre-trained networks into consideration (such as VGG or ResNet), it has a significant drawback: it neglects the spatial context along the z-axis (depth).

In neuro-oncology, a tumour is an inherently volumetric entity; it defines the anatomical boundaries with continuation across the sagittal, coronal and axial planes. When a 3D volume is processed as a series of 2D slices it frequently results in **"discontinuous predictions"**, which produce uneven and anatomically impossible 3D models where a lesion is found in one slice but overlooked in the adjacent one (Iftikhar *et al.*, 2025). To overcome this problem, the field has moved towards 3D CNNs, which simultaneously conduct convolutions on the entire (x, y, z) volume using volumetric kernels (such as $3 \times 3 \times 3$). This volumetric approach allows the model to learn complex spatial hierarchies and inter-slice connections, which is essential for precise segmentation of brain tumours using large-scale datasets like BraTS.

2.2.2 Evolution of Volumetric Segmentation Architectures

To handle the computational complexity of 3D data while conducting segmentation that is pixel-perfect, the research community has standardized the **"encoder-decoder"** architecture.

Building upon the success of 2D U-Net, Çiçek et al. (Çiçek *et al.*, 2016) introduced the 3D U-Net architecture, a fully convolutional network for dense volumetric image segmentation. A U-shaped symmetric model consists of two distinct pathways:

1. **The Contracting Path (Encoder)**: This pathway significantly considers feature extraction. It consists of max-pooling layers after recursive blocks of 3D convolutions. As data is processed through the encoder, the spatial resolution decreases while the number of features increases. This path allows the model to capture high-level conceptual data context ("where is the tumour present"), such as differentiating tumour tissues from healthy ones (Çiçek *et al.*, 2016).
2. **The Expanding Path (Decoder)**: To regenerate the segmentation map, the spatial resolution that was lost during encoding needs to be restored. This path reconstructs the feature maps back to their initial input dimensions using up-convolution (transposed convolution) layers, also providing precise localisation ("where is the tumour").

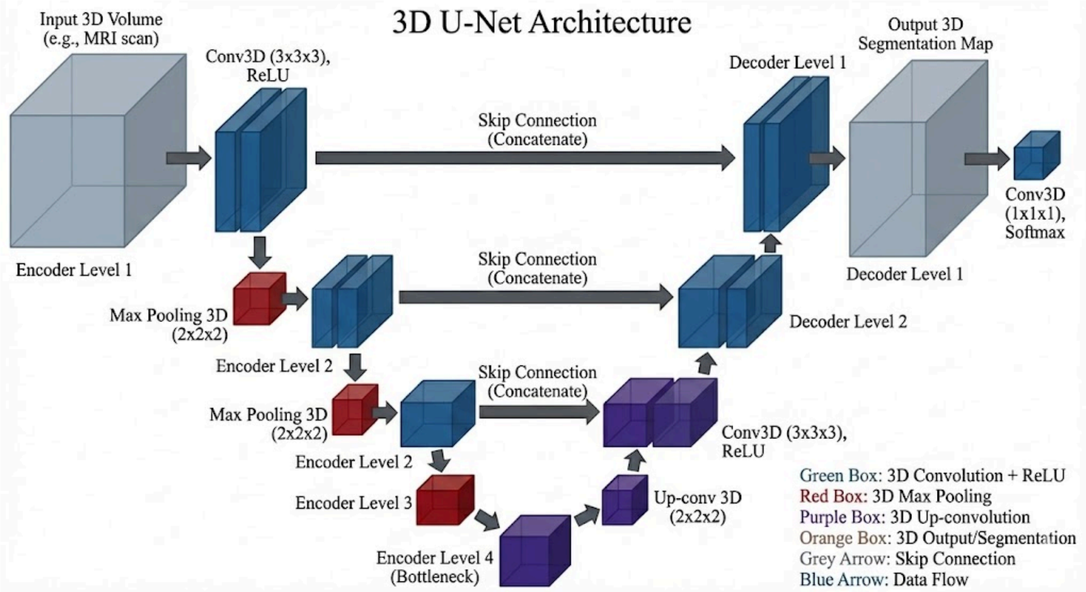


Figure 1: 3D U-Net Architecture

A critical innovation of the 3D U-Net introduced long-range skip connections. Deep neural networks often face difficulty acquiring fine-grained spatial information after applying multiple pooling operations. The 3D U-Net addresses this problem by concatenating high-resolution feature maps straight from the encoder to the appropriate decoder layers. According to Çiçek et al. (Çiçek *et al.*, 2016), this mechanism allows the network to incorporate deep semantic features with shallow details, with precise boundary delineation even when trained with sparse annotations.

Despite its success, the standard U-Net struggles with small lesions. Medical images often contain irrelevant background information such as non-considerable tissue in the brain, while the standard U-Net treats all the pixels in a feature map equally. To overcome this issue, researchers have integrated "attention mechanisms" into the U-Net architecture. As explained by Natekar et al. (Natekar, Kori and Krishnamurthi, 2020), the Attention U-Net incorporates **"attention gates"** into the skip connections.

These gates utilise the coarser signal from the gating vector (the deep layer) to filter the activations from the input signal (the shallow layer) before they are merged. This enhances the activations in the Region of Interest (ROI) and suppresses the irrelevant background regions using mathematical equations. This "soft-attention" mechanism improves the model's sensitivity to small, irregular lesions such as glioma subregions without requiring additional guidance or complex cropping preprocessing (Natekar, Kori and Krishnamurthi, 2020).

Parallel to the U-Net, Milletari et al. (Milletari, Navab and Ahmadi, 2016) proposed the V-Net, an optimised model specifically for volumetric medical data. V-Net is completely distinct from other models, incorporating **residual connections** (short-skip connections) blocks. This model resembles the ResNet philosophy, adding the input of a block to its output, allowing gradients to flow more smoothly during backpropagation. Compared to convolutional U-Net, this reduces the vanishing gradient issue and makes it easier to train much deeper 3D networks (Milletari, Navab and Ahmadi, 2016).

Furthermore, the V-Net research has presented a solution to one of the most enduring challenges in medical segmentation: class imbalance. In a dataset, tumour occupies less than 1% of the total volumetric pixels while the rest of the area is unaffected and healthy background. Standard objective functions like cross-entropy fail in this scenario, resulting in a model achieving accuracy of 99% by predicting "background irrelevant data" for every voxel. To overcome this, Milletari et al. (Milletari, Navab and Ahmadi, 2016) introduced the **Dice coefficient**, which optimises the model by considering the gap between the ground truth and the expected segmentation:

$$D = \frac{2\sum_i^N p_i g_i}{\sum_i^N p_i + \sum_i^N g_i}$$

Where the sums run through the N voxels, of the predicted binary volume $p_i \in P$ and the ground truth binary volume $g_i \in G$. This function can be improved by rewriting it in a differentiable form. As a result, it can be optimised with gradient descent. It can be computed with respect to each predicted voxel, allowing the network to improve the overlap between the predictions and ground truth (Milletari, Navab and Ahmadi, 2016).

The gradient:

$$\frac{\partial D}{\partial p_j} = 2\left[\frac{g_j}{\sum_i p_i^2 + \sum_i g_i^2} - \frac{2p_j\left(\sum_i p_i g_i\right)}{\left(\sum_i p_i^2 + \sum_i g_i^2\right)^2}\right]$$

2.3 Explainable AI (XAI) for Explainability in Medicine

2.3.1 The Need for Explainability in Clinical Practice

The integration of Artificial Intelligence (AI) into medical practice, especially in high-stakes domains like neuro-oncology, has been made more difficult by the opacity of modern Deep Learning (DL) models. Convolutional Neural Networks have demonstrated superior performance in segmentation-like tasks, yet their complex, non-linear decision-making remains inaccessible. This "black box" nature has become a barrier to model adoption, often referred to as the **"trust gap"** (Wen *et al.*, 2025).

Model prediction alone is insufficient in clinical practice; it must incorporate justification that follows medical knowledge. As highlighted by Neri et al. (Neri *et al.*, 2023), the ethical and legal framework governing medicine, such as the "right to explanation" under GDPR, demands that automated decisions be transparent and contestable. Clinicians must be able to identify the reason behind the model classification of a specific region as tumour-present or not to ensure patient safety (Iftikhar *et al.*, 2025).

Furthermore, XAI is not just a compliance tool but also a critical mechanism for model debugging and knowledge discovery. Researchers can detect "clever Hans" events, in which a model learns misleading connections (e.g., forecasting tumour based on a specific hospital watermark rather than anatomical pathology) (Hou *et al.*, 2024). When it comes to brain tumour segmentation, XAI refers to the validation of network focus on biologically relevant sub-regions such as the necrotic core or enhancing tumour (Natekar, Kori and Krishnamurthi, 2020).

2.3.2 Review of Voxel-Based XAI Techniques

The literature categorizes XAI approaches according to their scope and timing relative to model training. A primary distinction is made between ante-hoc (or "intrinsic") and post-hoc methods.

- **Ante-hoc:** Methods designed to be interpretable by their nature. For example, decision trees or linear regression models, where the relationships between them are explicit (Agrawal *et al.*, 2025). By integrating interpretable prototype blocks directly into the network architecture, recent advancements in "Self-Explainable AI (S-XAI)" plan to introduce transparency in deep learning (Hou *et al.*, 2024).
- **Post-hoc:** Methods designed for explainability after the model has been trained. They treat a model as a "black box" and try to improve and approximate its behaviour based on the given inputs and outputs. This method is preferred as standard for analysing medical imaging models like the 3D U-Net architecture to perform segmentation (Bhati, Neha and Amiruzzaman, 2024).

Further XAI methods are divided by their scope:

- **Global explanations:** Methods that explain the overall logic of the model across the entire dataset, such as feature importance in a model.

- **Local explanations:** Methods that explain the decision-making process of the model for specific inputs, such as the reason behind classifying a voxel region as tumour. Local explanation is important for clinical decision-making as they offer the patient-specific rationale for diagnosis and therapy planning (Iftikhar *et al.*, 2025).

For 3D medical imaging, the most common XAI techniques are those that can generate saliency maps, spatial representations of "feature importance". The three most prominent approaches are **Grad-CAM**, **LIME** and **SHAP**.

Gradient-Weighted Class Activation Mapping (Grad-CAM) has been recognised as the "golden standard" for visualising CNN decisions in medical imaging (Bhati, Neha and Amiruzzaman, 2024). Grad-CAM is model-agnostic and can be applied to any CNN model without retraining, compared to earlier methods that required architectural changes.

Mechanism: Grad-CAM utilises the gradient of the target concept (e.g., tumour) to produce a saliency map, which highlights the regions of the input image that contribute most to the prediction of the target class. It computes a weighted sum of the feature maps, where the weights represent the importance of each feature channel to the prediction. To eliminate the negative contributions, a Rectified Linear Unit (ReLU) is then applied, highlighting more specific regions that positively support the class of interest (Selvaraju *et al.*, 2017).

Application in 3D: Natekar et al. (Natekar, Kori and Krishnamurthi, 2020) successfully applied Grad-CAM to a 3D U-Net for brain tumour segmentation, as it was originally designed for 2D images. Their research showed that 3D Grad-CAM can localise tumours based on the aggregated gradients across volumetric feature maps. However, it has a critical limitation which is the resolution trade-off: because Grad-CAM uses feature maps from deep layers, the resulting heatmaps are often coarse and may bleed into surrounding tissue, reducing the voxel-level precision of the segmentation mask itself (Natekar, Kori and Krishnamurthi, 2020).

Local Interpretable Model-Agnostic Explanations (LIME) takes a different approach. LIME treats the model as a function only, without using internal gradients.

Mechanism: LIME modifies the model's predictions and observes the change by masking out random superpixels (segments). It then fits a simple, interpretable model to these disrupted samples to predict the model's behaviour locally around a specific image (Ribeiro, Singh and Guestrin, 2016).

Evaluation in 3D: While LIME is highly recommended for 2D image segmentation, its performance in 3D is computationally expensive. According to Agrawal et al. (Agrawal *et al.*, 2025), LIME requires thousands of forward passes to generate stable explanations. For data like 3D images which contain millions of voxels, generating valid superpixels and running sufficient distributions to achieve significant statistics is often too slow for real-time clinical workflows. Additionally, defining the "superpixels" in a 3D brain volume is quite difficult, frequently leading to unstable explanations that vary between runs.

SHAP (SHapley Additive exPlanations) provides a theoretically sound substitute based on cooperative game theory. Each feature (voxel) is given an "importance value" that represents its marginal contributions to the prediction, averaged over all possible combinations of features (Ribeiro, Singh and Guestrin, 2016).

Strengths and Weaknesses: SHAP provides the most mathematically consistent explanations, by conforming the summation of each feature's attributes to be equal to the model's output. However, it has extreme computational costs in high-dimensional spaces. "DeepSHAP," an approximation method, has been used to accelerate this process, but research suggests that SHAP values can mislead the explanation in deep networks, as they may violate axioms when features are highly correlated, which is often common in statistically coherent MRI data (Miró-Nicolau, Jaume-i-Capó and Moyà-Alcover, 2025).

2.3.3 Comparative Synthesis and the Fidelity Gap

All three methods have their strengths and weaknesses, but Grad-CAM remains the best performing method for 3D medical image segmentation because of its computational efficiency (only one backward pass is needed) and its capacity to make use of the spatial information present in CNN feature maps (Natekar, Kori and Krishnamurthi, 2020).

However, a critical "fidelity gap" exists, as existing measures frequently show significant discrepancy between the predicted and ground truth segmentation (Miró-Nicolau, Jaume-i-Capó and Moyà-Alcover, 2025). This discrepancy is exacerbated by the standard practice of analysing 3D attention maps as static 2D slices, a method that hides volumetric coherence and increases cognitive load for clinicians. Therefore, closing the "trust gap" calls for a paradigm shift towards immersive virtual reality, which provides the high-dimensional, intuitive visualisation needed to properly understand these deep volumetric insights, rather than simply algorithmic improvements.

2.4 Immersive Visualisation in Medical Imaging

The interpretation of volumetric medical data has historically been analysed on 2D screens, which causes a lot of cognitive stress. According to Khedir et al. (Khedir *et al.*, 2025), traditional slice-by-slice navigation requires clinicians to mentally reconstruct volumetric architecture. This process is time-consuming and prone to spatial errors when assessing the depth of the tumours relative to prominent brain regions. Even though 3D Convolutional Neural Networks (CNNs) can process this volumetric data, the representation of their outputs remains largely stuck in 2D.

Emerging research suggests that immersive visualisation such as Virtual Reality (VR) and Augmented Reality (AR) can bridge this gap. VR provides **stereoscopic depth perception** and **6-Degrees-of-Freedom (6DoF)** interaction which goes well beyond standard monitors. This allows users to view the "saliency maps" not as flat overlays, which are generated by XAI, but as volumetric clouds suspended in 3D space. The NeuroXAI framework has shown that immersive environments significantly improve clinicians' ability to localise pathology and understand complex neural connectivity compared to standard desktop viewers (Zeineldin *et al.*, 2022).

The application of VR extends well beyond passive viewing to active surgical planning. Defining tumour boundaries in neuro-oncology is critical. Recent advancements in Augmented Reality (AR) enable surgeons to simulate trajectories and visualise "risk maps" created by deep learning models before entering the operation theatre (Zeineldin, 2023; Khedir *et al.*, 2025). For situations like complex glioma, where 2D scans may not represent the tumour with key blood characteristics, this tool is a crucial resection corridor.

Additionally, the idea of interactive AI is essential to current medical systems. Static heatmaps (like standard Grad-CAM) offer **"take it or leave it"** explanations. VR environments, on the other hand, make **Human-in-the-Loop (HITL)** interaction easier. Immersive interfaces enable clinicians to interrogate the model, e.g., by digitally "erasing" a portion of the input volume to identify prediction changes (Orsmaa *et al.*, 2025). This interactivity transforms XAI from a static report into a dynamic conversation between the clinician and the AI, allowing the correction of model errors in real time (Orsmaa *et al.*, 2025).

To implement these immersive systems requires robust technological pipelines. Standard medical formats (DICOM/NIfTI) are not supported by native game engines like Unity. However, platforms like 3D Slicer have filled this bridge. The SlicerVR extension allows volume rendering of medical scans in VR headsets without any data conversion loss. Despite this, integrating XAI with VR still continues to be a major difficulty. Most pipelines concentrate on 2D slice-based visualisation, which fails to convey the volumetric feature importance and limits the clinical utility of XAI in more in-depth scenarios (Zeineldin *et al.*, 2022). There is a need for distinct workflows that can consume a 3D CNN's attention weights and render them as interactive volumetric objects, as proposed in frameworks like DeepIGN (Zeineldin, 2023).

2.5 Synthesis and Research Gap

2.5.1 The Disconnect Between Model Accuracy and Clinical Utility

After addressing and reviewing the three most significant chapters of this review, a distinct contrast has been established. Models like the 3D U-Net and its variants have achieved tremendous success in segmenting brain tumours (Çiçek *et al.*, 2016; Milletari, Navab and Ahmadi, 2016). On the other side, the opacity of the "black box" remains a critical barrier to clinical trust (Neri *et al.*, 2023). Although methods like Grad-CAM have successfully illuminated the internal logic of these networks (Natekar, Kori and Krishnamurthi, 2020), current implementations suffer from three major limitations that this project aims to address.

2.5.2 Identified Gaps in Current XAI Frameworks

Having global achievements in 3D XAI based on research such as NeuroXAI, there is still something which is called an **"interactivity gap"**; current systems use static visualisation rather than active user contribution (Zeineldin *et al.*, 2022). This passivity leads to a serious methodological error that confuses feature relevance with model confidence. As mentioned in the DeepIGN study (Zeineldin, 2023), a network may exhibit high focal attention on a specific region while displaying high epistemic uncertainty. This could give a confident result but incorrect false positives. As a result, existing 3D XAI methods do not provide the required mechanisms for **interactive correction or uncertainty quantification**, leaving a functional gap where the user cannot distinguish between a confident prediction and a statistical guess, or change the model's focus (Abyasa and Rahmania, 2025).

Furthermore, the utility of these volumetric insights is severely constrained by the "cognitive friction" of traditional visualisation mediums. Khedir et al. (Khedir *et al.*, 2025) mentioned that analysing 3D volumetric pathology via slice-by-slice 2D navigation increases significant burden to mentally reconstruct spatial relationships from disconnected images. The mismatch between the 3D nature of the data and the 2D nature of the display limits the interpretability of the feature maps, as crucial depth signals are lost in translation. Therefore, a clear research gap exists for a unique framework that not only generates uncertainty-aware explanations but also represents them in an immersive virtual environment, to clarify whether or not engagement significantly lowers cognitive load compared to conventional medical workflows.

2.6 Conclusion

This literature review establishes that while 3D deep learning architectures, particularly the 3D U-Net, have achieved superior success in volumetric tumour segmentation, their deployment is critically hindered by the "black box" paradox (Neri *et al.*, 2023). Clinical workflows still remain detached even if explainable methods like Grad-CAM provide mathematical solutions for transparency. The reliance on static 2D slice-by-slice visualisation fails to convey the complex spatial resolution of 3D pathologies and is hindered by a significant cognitive load that limits clinical trust (Natekar, Kori and Krishnamurthi, 2020).

Consequently, this research focusing on bridging the "trust gap" requires a paradigm shift from passive observation to an interactive environment. By developing a virtual reality environment that incorporates uncertainty quantification with real-time feedback, this project aims to "illuminate the black box" effectively. This innovative method will ensure that high-performance AI is not just accurate but also understandable and useful for medical practice.

Chapter 3: Methodology

3.1 Introduction

This chapter delineates the methodological framework for developing an Explainable AI (XAI) system capable of both accurate brain tumour segmentation and transparent clinical decision-making. The central objective is to address the “black-box” limitation common in medical imaging AI by transforming a complex deep learning model into an interpretable system. This system aims to achieve high segmentation accuracy while providing clinicians with actionable insights into the underlying reasoning behind each prediction.

To ensure transparency, the framework integrates six complementary XAI methodologies, each validated against ground-truth annotations to confirm their clinical relevance. At the core of this system is SegResNet, a 3D residual encoder-decoder architecture designed to process full volumetric MRI data rather than individual 2D slices. This volumetric approach enables the model to effectively capture the spatial hierarchies across three critical tumour sub-regions: the Whole Tumour (WT), Tumour Core (TC), and Enhancing Tumour (ET).

The research design follows a hybrid Agile/CRISP-DM methodology, strategically combining the systematic rigor required for medical data mining with the iterative flexibility necessary for deep learning experimentation. This dual approach ensures methodological robustness appropriate for safety-critical healthcare applications while accommodating the evolutionary nature of neural network development, which transitioned from initial rapid prototyping on a 250-patient subset to full-scale training on the complete BraTS corpus.

The remainder of this chapter is structured as follows: Section 3.2 outlines the overall research approach and rationale; Section 3.3 details the technical implementation, including data preprocessing, model architecture, and XAI methods; Section 3.4 describes the research design and evaluation strategy; Section 3.5 discusses technical challenges and methodological limitations; and Section 3.6 addresses ethical considerations and data governance.

3.2 Approach

3.2.1 Overall Approach and Rationale

This study creates a three-stage pipeline using the BraTS 2023 dataset, which consists of about 1,250 multi-modal Magnetic Resonance Imaging (MRI) patient cases: (1) training a SegResNet 3D Convolutional Neural Network (CNN) for volumetric tumour segmentation; (2) integrating multiple XAI techniques to illuminate model decision-making; and (3) exporting results into a Virtual Reality (VR) compatible format for immersive clinical evaluation. This volumetric methodology captures the true three-dimensional character of brain tumours, producing more anatomically accurate segmentations of the Whole Tumour, Tumour Core, and Enhancing Tumour areas, in contrast to 2D slice-by-slice methodologies that sacrifice inter-slice spatial context (Milletari, Navab and Ahmadi, 2016; Çiçek *et al.*, 2016).

SegResNet was selected as the primary architecture due to its residual connections, which provide robustness against vanishing gradients and enable effective training of deep networks (He *et al.*, 2016). The architecture offers an optimal balance between performance and computational complexity, making it suitable for medical imaging applications where training data may be limited and model interpretability is essential.

The rationale for implementing multiple XAI methods stems from their complementary strengths. Gradient-based methods (Grad-CAM, Guided Backpropagation, Input × Gradient) provide different perspectives on feature importance, while perturbation-based approaches (Occlusion Sensitivity) offer model-agnostic explanations. Monte Carlo Dropout adds uncertainty quantification, enabling identification of regions where the model lacks confidence, a critical capability for clinical decision support.

3.2.2 Agile CRISP-DM Methodology and Its Use

Deep learning model training is very exploratory and unpredictable, making traditional waterfall development lifecycles inappropriate. A more flexible approach is required due to the intricacy of neural network designs, as well as the requirement for substantial hyperparameter adjustment and iterative improvement. The Cross-Industry Standard Process for Data Mining's (CRISP-DM) structured phases are combined with Agile development's rapid iteration and evolutionary prototyping concepts in this study's hybrid Agile/CRISP-DM approach.

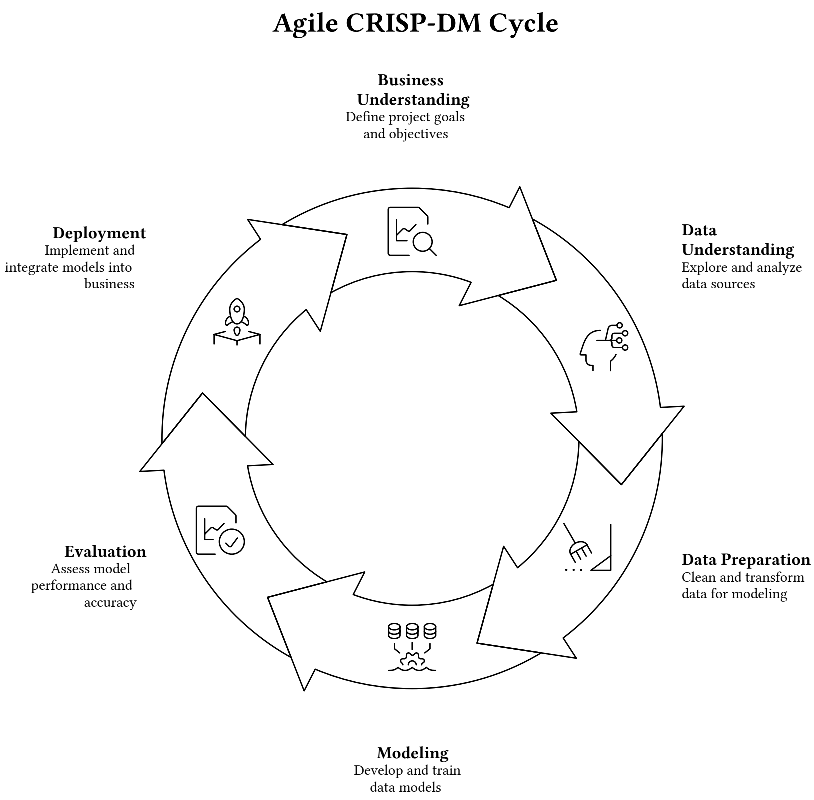


Figure 2: The Agile CRISP-DM methodology employed in this research, showing the iterative cycles between the six core phases.

Figure 2 illustrates the six interconnected phases of the methodology, with dashed arrows indicating the iterative nature of the development process. Each phase feeds into the next, but the Agile approach allows for rapid cycling back to earlier phases when issues are discovered or improvements are identified.

The framework, which was created using an iterative CRISP-DM process, was continuously validated against strict clinical criteria (Dice, HD95, IoU, sensitivity, and specificity) while containing Explainable AI components that were verified by saliency overlap and uncertainty quantification. Although the system is still an experimental research prototype awaiting clinical deployment, the pipeline produces standardised NIfTI outputs, containing segmentations, attention heatmaps, and variance arrays for direct 3D Slicer (Fedorov *et al.*, 2012) VR visualisation.

The Agile CRISP-DM approach ensured an organized, rapid-iteration development loop while maintaining the stringent robustness required for safety-critical medical machine learning components. Regular sprint reviews allowed for course correction, and the modular architecture enabled independent validation of each component.

3.3 Implementation

3.3.1 Data Acquisition and Preprocessing

The BraTS 2023 dataset, provided through the MICCAI Brain Tumour Segmentation Challenge, serves as the foundation for this research. The dataset comprises approximately 1,251 patient cases, each containing four co-registered MRI modalities: T1-weighted (T1), T1-weighted with Gadolinium contrast enhancement (T1c), T2-weighted (T2), and T2 Fluid-Attenuated Inversion Recovery (FLAIR). All volumes have been resampled to an isotropic resolution of 1 mm^3 and skull-stripped.

The tumour labels in BraTS follow a standard annotation scheme where each voxel is classified as background (0), necrotic tumour core (1), peritumoural edema (2), or enhancing tumour (3). For this research, these labels are converted into three binary channels representing clinically relevant regions: Whole Tumour (WT = necrosis + edema + enhancing), Tumour Core (TC = necrosis + enhancing), and Enhancing Tumour (ET = enhancing only). The preprocessing pipeline optimises raw MRI volumes for efficient 3D model training through four core operations: foreground cropping reduces spatial dimensions from 240^3 to 160^3 voxels by removing extraneous background; channel-first formatting restructures data to [4, 160, 160, 160] for PyTorch compatibility; per-channel z-score normalization standardizes intensities across modalities using only brain tissue statistics; and random spatial augmentation (training only) applies geometric and intensity transforms - such as random flips, adding noise, etc. - to enhance model robustness.

3.3.2 SegResNet Model Architecture

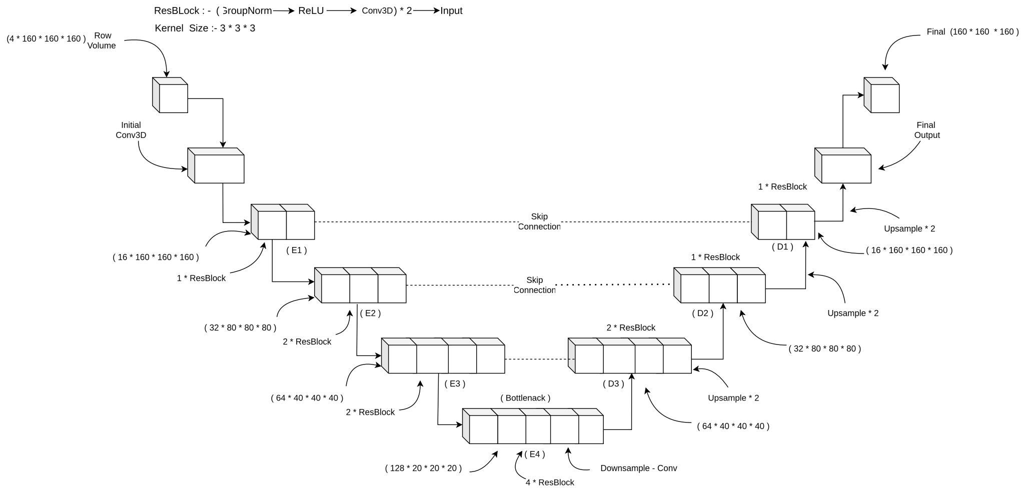


Figure 3: The SegResNet architecture, showing the encoder-decoder structure with residual connections and skip connections.

The implementation utilises SegResNet, a 3D encoder-decoder architecture designed for volumetric segmentation that processes entire MRI volumes to capture global spatial dependencies. The encoder progressively downsamples input modalities through four levels, increasing filters from 32 to 256 channel bottleneck, using strided convolutions that reduce resolution from 160^3 to 20^3. Every layer is constructed from Residual Blocks (ResBlocks) featuring 3×3×3 kernels, GroupNorm, and ReLU activations. These blocks utilize skip connections to prevent vanishing gradients and ensure training stability by learning residual mappings rather than opaque transformations.

The bottleneck layer offers a compressed, globally-aware representation well-suited for Grad-CAM analysis. The decoder reverses the encoder via trilinear upsampling and skip connections, recovering fine-grained spatial detail for the final 160^3 outputs. To mitigate severe class imbalance particularly the Enhancing Tumour (ET) region at merely 5% volume, the model employs DiceFocalLoss ($\gamma = 2.0$) to emphasize misclassified voxels. A dropout rate of 0.1 serves dual purposes: regularization and enabling MC Dropout for uncertainty quantification in clinical review.

3.3.3 Evolutionary Prototyping Phase

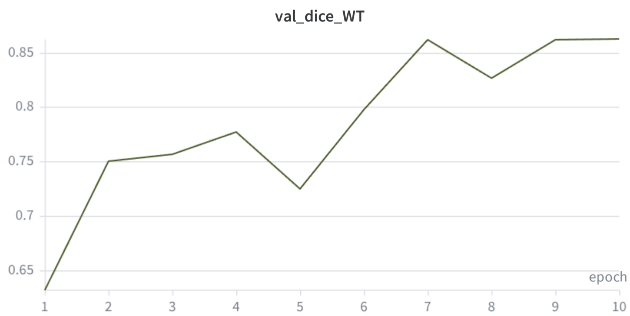
Before committing to full-scale training, an evolutionary prototyping phase was conducted using a 250-patient subset randomly sampled from the BraTS training data. This phase served multiple purposes: validating the complex 3D data pipeline, rapidly testing candidate architectures, and verifying hardware memory limits. The 250-patient prototype subset was drawn from the same BraTS 2023 corpus used for full-scale training. As the full training utilises the entire 1,251-patient dataset with a fresh deterministic split (seed = 42), prototype test patients may appear in the final training set. This does not constitute leakage for the final reported metrics, which are evaluated exclusively on the held-out 126-patient test split that was never exposed during full-scale training. The initial prototype was implemented using two NVIDIA

GeForce RTX 2080 Ti GPUs (11 GB VRAM each), which allowed for training across 10 epochs in approximately 1 hour 2 minutes. The dataset was split into 200 training cases, 25 validation cases, and 25 test cases for this phase.

The prototype results demonstrated that SegResNet provided the optimal performance-to-complexity ratio among the tested architectures. The 250-patient subset achieved promising results on the prototype test set with Dice scores of 0.80 for the Whole Tumour (WT), 0.55 for the Tumour Core (TC), and 0.34 for the Enhancing Tumour (ET), confirming the viability of the approach for full-scale implementation. After validating this core functionality, the system was refactored into its final modular, configuration-driven PyTorch framework.

(a) Training Loss

(b) Batch Loss



(c) Validation Whole Tumour (WT) Dice

Figure 4: Training metrics for the 250-patient evolutionary prototype.

3.3.4 Implementation Pipeline

The implementation follows a modular architecture designed for reproducibility and extensibility, organized into several key components. YAML-based configuration files manage all hyperparameters and paths, supporting accessible parameter sweeps across the experiments. The core data pipeline (`src/data`) manages dataset scanning, patient splitting, and MONAI transform pipelines, while the model factory (`src/models`) dynamically instantiates model architectures, loss functions, and optimisers. Operating in tandem, the training framework (`src/training`) orchestrates the complete training lifecycle, integrating automatic mixed precision, automated checkpointing, and tracking via Weights & Biases. Finally, the explainability aspect is controlled by the XAI suite (`src/xai`), which standardizes the generation of saliency maps and uncertainty metrics, supported directly by the evaluation tools (`src/evaluation`) responsible for computing both segmentation and novel XAI-specific metrics and exporting them to CSV configurations.

The end-to-end workflow proceeds through seven stages: (1) configuration loading and data initialization, (2) model training with validation, (3) test set inference, (4) XAI map generation for all methods, (5) metric computation and CSV export, (6) NIfTI volume export for 3D Slicer (Fedorov *et al.*, 2012), and (7) VR visualisation preparation.

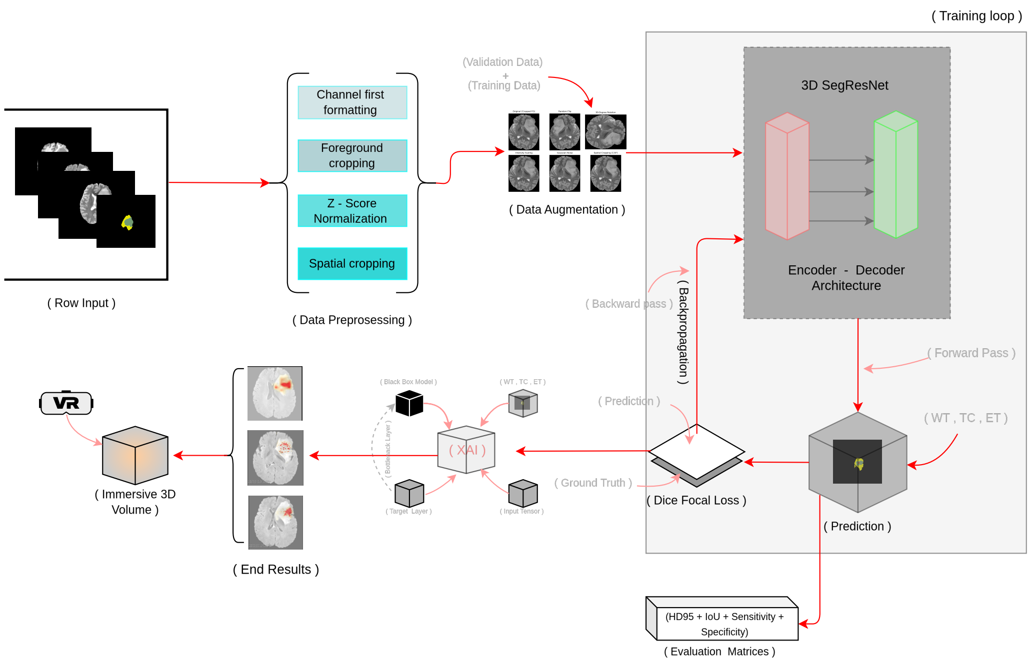


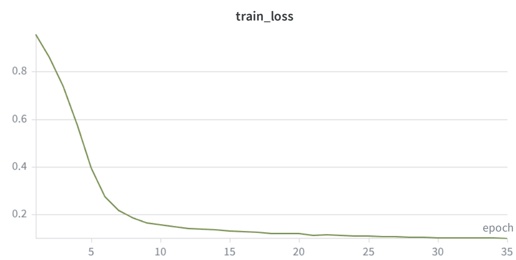
Figure 5: The implementation pipeline, showing the modular architecture and end-to-end workflow.

3.3.5 Full-Scale Training Setup

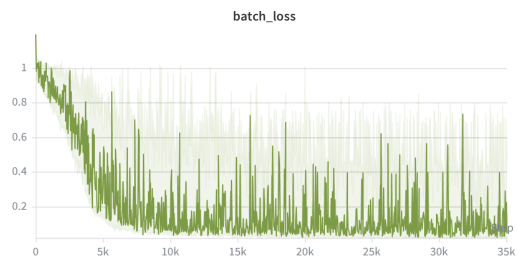
The final training configuration was optimised for a single high-memory GPU (RTX 5880 Ada, 48 GB VRAM) with Automatic Mixed Precision (AMP) enabled to maximize training efficiency. Key hyperparameters include:

Parameter	**Value**
Batch Size	1 (limited by 3D volume memory)
Epochs	35
Learning Rate	5×10^{-5}
LR Scheduler	Cosine Annealing
Optimizer	AdamW (weight decay 1×10^{-5})
Loss Function	DiceFocalLoss
Focal Gamma	2.0
Mixed Precision	Enabled

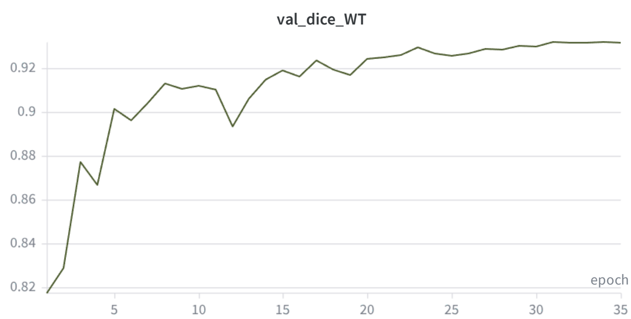
Table 1: SegResNet training hyperparameters for BraTS 2023 full dataset.



(a) Training Loss



(b) Batch Loss



(c) Validation Whole Tumour (WT) Dice

Figure 6: Training metrics for the full-scale SegResNet model (1,251 patients).

The dataset was split into 1,000 training cases, 125 validation cases, and approximately 126 test cases. The validation set was used for hyperparameter tuning and early stopping, while the test set was reserved for final evaluation and XAI analysis. All splits were performed deterministically using a fixed random seed (42) to ensure reproducibility. These results are reported from a single deterministic test split; confidence intervals are not available without cross-validation, which was computationally infeasible at this scale.

Training progress was monitored using Weights & Biases (W&B) (Biewald, 2020) for experiment tracking, with metrics logged at every iteration. Best model checkpoints were selected based on validation Dice score, and the final model was evaluated on the held-out test set to report unbiased performance estimates.

3.3.6 XAI Module Implementation

The XAI framework implements six complementary explanation techniques, each providing unique insights into model decision-making:

Grad-CAM (Gradient-weighted Class Activation Mapping) computes per-feature-map importance weights by analyzing gradients flowing into the final convolutional bottleneck layer. This provides class-discriminative localization indicating regions critical for predicting a specific tumour sub-region. For a target class c and feature map k, the importance weight α_k^c is calculated by global-average-pooling the gradients across the spatial dimensions:

$$\alpha_k^c = \frac{1}{Z} \sum_x \sum_y \sum_z \frac{\partial y^c}{\partial A_{x,y,z}^k}$$

where y^c is the spatially-averaged output logit, $A_{x,y,z}^k$ represents the feature map activation, and Z is the total number of spatial elements. The gradient term $\frac{\partial y^c}{\partial A_{x,y,z}^k}$ mathematically represents the sensitivity or "importance" of the target class score y^c with respect to a specific pixel in the k-th feature map. By computing this partial derivative via backpropagation, the model quantifies exactly how much a change in that specific convolutional feature would objectively influence the final class probability. A linear combination of these feature maps is then computed, followed by a ReLU activation to preserve only features that positively support the target class:

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right)$$

The resulting heatmap is upsampled to the original input resolution, providing class-specific but coarse spatial localization.

Guided Backpropagation (GBP) adds an additional guidance signal to the standard backpropagation algorithm to highlight areas where significant features are present at a highly detailed, voxel-level scale. When passing gradients backward through a ReLU unit at layer l, GBP suppresses the flow of negative gradients by enforcing two conditions: the original forward activation must have been positive, and the incoming backward gradient must also be positive. The guided gradient R_i^l propagated to neuron i is formulated as:

$$R_i^l = \left(f_i^l > 0\right) \cdot \left(R_i^{l+1} > 0\right) \cdot R_i^{l+1}$$

where f_i^l is the forward activation and R_i^{l+1} is the incoming gradient from the higher layer. Because both conditions act as binary indicator functions, purely positive signal paths are isolated. This generates a full-resolution (160^3) saliency map with sharp edges and fine detail, although it is not inherently class-discriminative.

Guided Grad-CAM fuses the complementary strengths of the previous two methods to achieve visualisations that are both high-resolution and class-specific. It masks the high-frequency pixel details produced by GBP using the coarse, class-discriminative localization of Grad-CAM through element-wise multiplication:

$$L^c_{\text{Guided Grad-CAM}} = L^c_{\text{Grad-CAM}} \cdot L_{\text{Guided Backpropagation}}$$

This process effectively masks out the fine-grained features that are irrelevant to the target class, resulting in high-resolution attributions that accurately highlight the structural features directly responsible for the model's prediction.

Input × Gradient (LRP Proxy) operates on a first-order Taylor decomposition, attributing a relevance score to each input voxel proportional to both its magnitude and the model's sensitivity to it. True layer-wise relevance propagation (Bach et al., 2015) (Bach *et al.*, 2015) requires implementing conservation rules (ε-rule, $\alpha\beta$-rule) through every network layer individually. Due to the architectural complexity of SegResNet - which encompasses residual skip connections and GroupNorm layers that violate the strict assumptions of standard LRP propagation rules - developing manual backward hooks for every layer block is prohibitively complex. Therefore, the implementation utilises the Input × Gradient approximation, a tractable proxy that serves as the closest feasible approximation to ε-LRP for this architecture. The relevance R_i assigned to the i-th voxel of the input x_i concerning the spatial model prediction score $f(x)$ is formulated as:

$$R_i \approx x_i \cdot \frac{\partial f(x)}{\partial x_i}$$

This approach directly circumvents architectural bottlenecks, yielding an ultra-high resolution (160^3) relevance map that quantifies which specific structural features explicitly drove the network's prediction.

Occlusion Sensitivity serves as a purely empirical, model-agnostic perturbation technique. It bypasses internal gradient calculations altogether, instead physically testing the 3D CNN's structural reliance by obscuring data. The implementation employs a 3D sliding window defined by a $16 \times 16 \times 16$ spatial occluding patch that moves across the target volume with a stride of 8 voxels. At each step, the occluded region within the input MRI modalities x is replaced with a baseline value of zero, creating an occluded volume x_{occluded}. For a target class c, the spatial sensitivity score S_c evaluated at the occluding window's center coordinate (i, j, k) measures the absolute drop in the model's confidence f_c:

$$S_c(i, j, k) = f_c(x) - f_c(x_{\text{occluded}})$$

Because the sliding window utilises a stride of 8 across a 160^3 input space, the resulting sensitivity map is initially evaluated at a dimension of 20^3. It is subsequently reconstructed and upsampled back to 160^3 via trilinear interpolation. The confidence score f_c is computed as the spatial mean of class logits rather than their sum, ensuring computational stability across volumes of varying tumour size and maintaining consistency with Grad-CAM's score function, which also spatially averages logits. A consequence of this choice is that small subregions (e.g., Enhancing Tumour at 5% volume) produce attenuated sensitivity scores because their occlusion has minimal impact on the spatial mean - a limitation acknowledged in the results.

Monte Carlo (MC) Dropout quantifies confidence rather than providing structural attribution. Because traditional inference is deterministic, this implementation introduces test-time stochasticity by forcing SegResNet's built-in dropout layers (at a probability of $p = 0.1$) to remain active during the evaluation phase. Generating $N = 20$ stochastic forward passes for an identical volumetric input produces a probability distribution over the predicted segmentations. Given the predicted output probability p_n from the n-th forward pass, the stabilized mean prediction $\bar{p}$ and its corresponding voxel-wise predictive variance (uncertainty map) σ^2 are evaluated as:

$$\bar{p} = \frac{1}{N} \sum_{n=1}^{N} p_n$$

$$\sigma^2(x, y, z) = \frac{1}{N} \sum_{n=1}^{N} \left(p_{n(x,y,z)} - \bar{p}(x, y, z)\right)^2$$

Where high variance occurs, it highlights “brittle” transition zones or anatomically ambiguous boundaries where the network remains unconfident. Comparing Input × Gradient attributions against these uncertainty maps isolates clinically critical structural dependencies that carry significant diagnostic risks.

All XAI outputs are saved as NIfTI volumes organized per patient and per method, ready for loading into 3D Slicer (Fedorov *et al.*, 2012) or VR visualisation tools. The implementation ensures spatial alignment between saliency maps and ground truth labels by computing metrics inline during generation, using the same preprocessed coordinate space.

3.4 Research Approach

3.4.1 Research Design

This study employs applied, quantitative research using secondary medical imaging data. The experimental design evaluates one primary architecture (SegResNet) combined with six XAI techniques, measuring both segmentation performance and explanation validity against ground truth annotations.

The experimental unit is a single BraTS patient case. Independent variables include the choice of XAI method and the tumour sub-region being evaluated (WT, TC, ET). Dependent variables encompass segmentation metrics (Dice, HD95, IoU, Sensitivity, Specificity), XAI metrics (Pointing Game, Coverage, IoU, Weighted Dice), and uncertainty statistics (UAR, boundary ratios).

3.4.2 Evaluation Strategy

Model performance and internal reliability are evaluated systematically. To ensure clinical interpretability, the quantitative assessment is divided into mathematical evaluations for structural segmentation accuracy and subsequent evaluations for XAI attribution and uncertainty.

1. Segmentation Performance Metrics The core segmentation performance of SegResNet is benchmarked against the clinically-annotated test set using the following spatial metrics:

Metric Name	Formula	Description / Function
Dice Score (DSC)	$\frac{2\|A \cap B\|}{\|A\| + \|B\|}$	Measures volumetric overlap between prediction (A) and ground truth (B) from 0 to 1. Primary BraTS benchmark.
HD 95th Percentile	$P_{95}(d(a, B) \cup d(b, A))$	Measures 95th percentile surface boundary error in millimeters. Crucial for assessing margin precision.
Intersection over Union	$\|A \cap B \frac{\|}{\|} A \cup B\|$	A stricter spatial overlap metric (Jaccard index) penalizing localized false positives more heavily than Dice.
Sensitivity (Recall)	$\frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$	The true positive rate. Measures the fraction of actual tumour voxels correctly identified to prevent missed diagnoses.
Specificity	$\frac{\mathrm{TN}}{\mathrm{TN} + \mathrm{FP}}$	The true negative rate. Measures reliability in excluding healthy tissue, directly preventing false alarms.

Table 2: Summary of quantitative metrics used for evaluating 3D volumetric segmentation accuracy.

2. Explainability and Uncertainty Metrics Beyond structural accuracy, the generated gradient-based visual attributions and the MC Dropout variance maps (σ^2) are quantitatively assessed utilising custom validation metrics:

Metric Name	Formula	Description / Function
Pointing Game (PG)	$\begin{cases} 1 & \text{if} \arg\max(L) \in G \\ 0 & \text{otherwise} \end{cases}$	Binary test returning 1 if the highest activation peak in the saliency map L falls inside the tumour region G.
Saliency Coverage	$\frac{\sum_{x \in G} L(x)}{\sum_x L(x)}$	Calculates the percentage of total salient relevance mass concentrated inside the true tumour boundary.
Saliency IoU	$\lvert L_{\text{thresh}} \cap G \rvert / \lvert L_{\text{thresh}} \cup G \rvert$	Evaluates shape overlap between a thresholded Boolean heatmap (L_{thresh}) and the ground truth mask (G).
Weighted Dice	$\frac{2 \sum L(x) \cdot G(x)}{\sum L(x) + \sum G(x)}$	Treats continuous heatmaps as soft predictive weights, modeling organic relevance without binary thresholds.
Uncertainty Area Ratio	$\frac{\sum_{x \in G} \sigma^2(x)}{\sum_x \sigma^2(x)}$	Computes the ratio of predictive uncertainty (variance σ^2) trapped inside the pathogenic ROI versus total volume.
Saliency-Uncertainty	$\frac{\text{Cov}(L, \sigma^2)}{\sigma_L \sigma_{\sigma^2}}$	Pearson correlation isolating "brittle" behaviors where high relevance (L) intersects with high variance (σ^2).

Table 3: Summary of metrics utilized to evaluate XAI map alignment and model uncertainty.

3.4.3 Link to Research Questions

This methodological design directly addresses the research questions posed in Chapter 1:

- **Segmentation Accuracy** is measured by traditional metrics (Dice, HD95), answering whether the model achieves clinically acceptable performance.
- **Model Attention Alignment** is assessed by XAI metrics (Pointing Game, Coverage), answering whether the model "looks at the right place for the right reasons."
- **Reliability Characteristics** are quantified by uncertainty metrics, answering where the model is confident versus uncertain.

By combining these evaluation dimensions, this research can argue not only that the model is accurate, but also that its decision-making process is clinically interpretable and trustworthy.

3.5 Challenges and Limitations

This research navigated several primary technical and methodological constraints:

Computational and Architectural Constraints: The memory-intensive nature of 3D CNNs mandated a limited batch size (1) and a 160^3 volumetric ROI, affecting training efficiency (approx. 24 hours for 35 epochs). XAI generation proved exceptionally demanding; Occlusion

Sensitivity required approximately 1,000 forward passes per patient. Architecturally, integrating gradient-based XAI methods required strict hook isolation. For instance, simultaneous Grad-CAM and Guided Backpropagation hooks inherently corrupt each other's gradients, demanding sequential, per-method computation workflows.

Generalizability and Domain Shift: Methodologically, relying exclusively on an isolated dataset (BraTS 2023) without external cross-institutional validation restricts the clinical generalisation of the model. Its segmentation performance may degrade due to domain shift if exposed to MRI scans from alternate hospital scanners, protocols, or distinct neuropathologies.

XAI Construct Validity: Finally, while metrics like the Pointing Game and Weighted Dice effectively operationalize interpretability mathematically, they serve merely as academic proxies for clinical utility. Furthermore, the Virtual Reality (VR) application acts primarily as an exploratory visualisation target; comprehensive studies involving practicing radiologists remain critical to validating the actual diagnostic efficacy of these immersive explanation maps.

3.6 Ethical Considerations

3.6.1 Data Privacy and Governance

This research uses only de-identified BraTS data that has been stripped of all personally identifiable information. Data handling practices include local secure storage with restricted access, no attempt at re-identification, and adherence to dataset usage terms. The BraTS challenge operates under appropriate ethical approvals for secondary use of medical imaging data in research.

3.6.2 Clinical Safety and Responsible Use

It is essential to emphasize that this system is a research prototype and has not been approved for clinical decision-making. The intended use is as a decision-support and research tool to explore model behavior, not as an autonomous diagnostic system. Risks include over-reliance on AI outputs and potential misinterpretation of saliency maps or uncertainty maps by non-expert users.

Clear documentation accompanies all outputs explaining what XAI maps represent and what they do not. The system is designed to augment, not replace, clinical expertise.

3.6.3 Bias and Generalisation

The BraTS dataset, while large and diverse, may contain biases related to geography, scanner types, and pathology distribution. Limited external validation means performance may change on under-represented populations. Future work should include broader, more diverse datasets to ensure equitable performance across demographic groups.

3.6.4 Ethical Use of VR for Medical Imaging

Immersive visualisation may amplify visual cues (e.g., bright saliency hotspots) and affect clinician perception. Safeguards for future deployment include clear documentation on XAI map interpretation, training materials for clinicians, and emphasis on combining AI insights with expert judgment rather than relying solely on automated outputs.

Chapter 4: Results

This chapter presents a comprehensive appraisal of the 3D Convolutional Neural Network (CNN) (SegResNet) and its efficacy. The results are organised into three sequential analyses: an evaluation of the model's predictive segmentation performance, the interpretability of its decisions via Explainable Artificial Intelligence (XAI) techniques, and the subsequent visualisation of these complex metrics within an immersive Virtual Reality (VR) framework.

4.1 Prediction and Segmentation Performance

The first phase of this study involved developing a neural network architecture capable of accurately localizing brain tumour subregions, serving as a prerequisite for subsequent explainability analyses. Given the substantial histopathological heterogeneity and complex morphological variations characteristic of neoplastic lesions (actual tumour), establishing a robust segmentation backbone was imperative to ensure the validity of downstream interpretability methods.

4.1.1 Progression and Metric Analysis (Baseline vs. Final)

This section presents a comparative analysis of model iterations developed through the Agile-based methodology previously described. Beginning with a baseline prototype (Version 1) trained on minimal data and culminating in a fully-specified model trained on 1,251 patients from the BraTS cohort, the evaluation metrics reveal substantial performance divergences between these developmental phases.

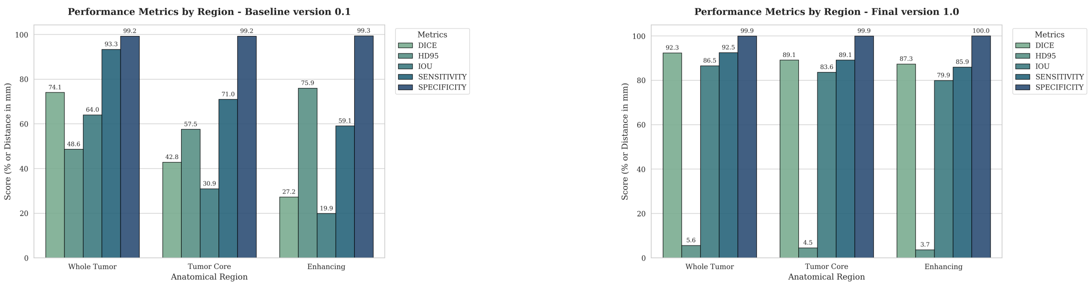


Figure 7: Performance comparison between the Baseline Prototype (left) and the Final Model trained on 1,251 patients (right). Note the massive improvement in the ET metric.

Data Scaling Comparison To understand how dataset volume directly influences performance, we compared the Dice scores across three versions: the initial baseline, an intermediate training run of 250 patients, and the final run utilising the complete 1,251-patient dataset.

	Dice Score			HD95 (mm)		
Region	**Baseline**	**250 Pts**	**Final**	**Baseline**	**250 Pts**	**Final**
WT	0.741	0.908	0.923	48.63	7.23	5.60
TC	0.428	0.837	0.891	57.54	15.34	4.54
ET	0.272	0.742	0.873	75.91	15.25	3.66

Table 4: Test set Dice and HD95 progression across data scaling stages. Dice quantifies volumetric overlap (higher is better) while HD95 measures boundary error in millimetres (lower is better).

Table 4 presents segmentation metrics across training data scales (45 to 1,251 patients), utilising Dice coefficients for volumetric overlap and 95th Percentile Hausdorff Distance (HD95) for boundary delineation. Whole Tumour (WT) segmentation exhibited asymptotic performance (Dice 0.908 to 0.923), suggesting efficient learning of macroscopic structures from limited data. In contrast, the Enhancing Tumour (ET) characterized by fragmented, heterogeneous morphology demonstrated substantial dependency on training volume, improving from 0.272 to 0.873. Corresponding boundary refinement was equally pronounced: ET HD95 decreased from 75.91 mm to 3.66 mm, progressing from clinically unusable deviations to sub-voxel precision requisite for safe neurosurgical planning.

Computational vs. Performance Trade-off

The escalation from 250 patients to 1,251 patients carried substantial hardware and temporal costs. Training iterations lasted exponentially longer on the GPU infrastructure. However, as the table indicates, the added computation was unequivocally justified. While general regions (WT) hit a point of diminishing returns in both Dice and HD95, the deeper feature extraction required to confidently segment the complex non-linear Enhancing Tumours and to tighten their boundary precision required the full breadth of the training corpus.

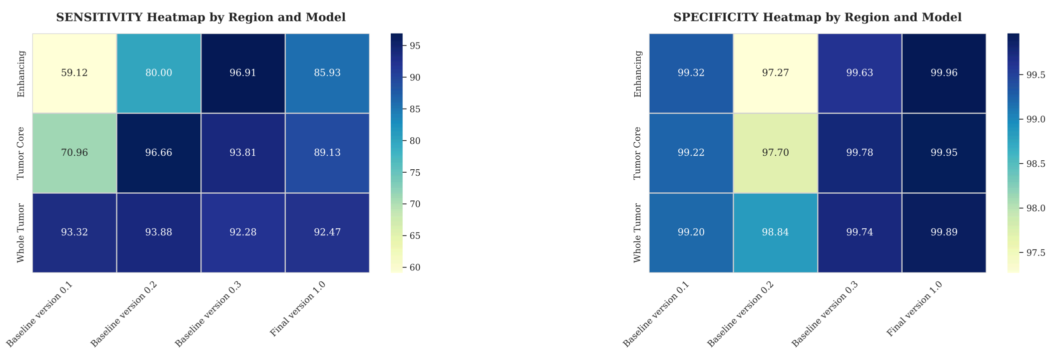


Figure 8: Sensitivity (left) and Specificity (right) heatmaps across model versions. Sensitivity measures the proportion of true tumour voxels correctly identified, while Specificity reflects the model's ability to correctly exclude healthy tissue.

4.1.2 Qualitative Visual Evaluation (2D & 3D Comparisons)

Morphological fidelity is still necessary for therapeutic value, even though quantitative metrics demonstrate computational validity. Precise spatial localisation of complex tumour borders is necessary for accurate subregion delineation. In order to confirm spatial concordance and anatomical accuracy, SegResNet predictions are compared to expert-annotated ground truth using multi-planar (2D) and volumetric (3D) visual evaluations.

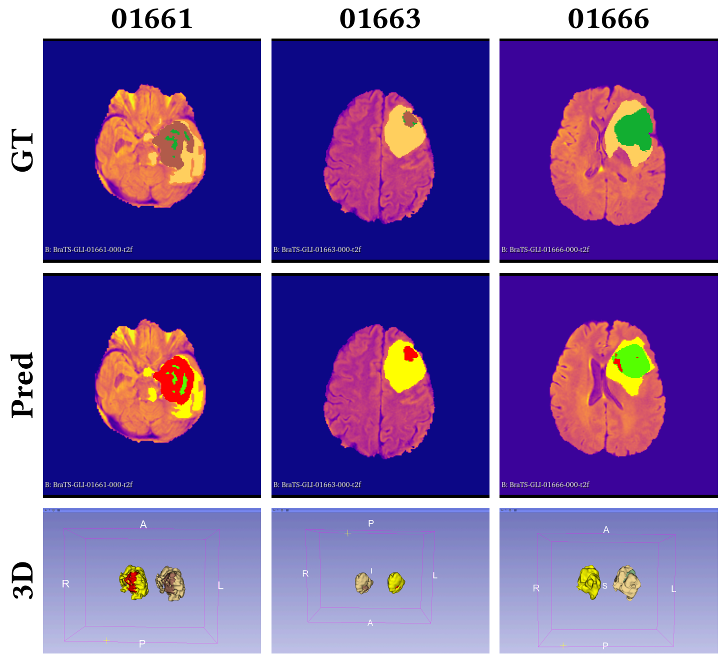


Figure 9: Qualitative comparison of patients 01661, 01663, and 01666. Row 1: Ground Truth (GT). Row 2: Model Prediction (Pred). Row 3: 3D volumetric rendering of the predicted segmentation.

2D Alignment Analysis When examining the 2D planes, the model's predicted boundary layers flawlessly align with the True Ground Truth masks. As established, the 3.66 mm precision in HD95 manifests visually here: the predicted layers perfectly hug the edges of the actual tumour on the base Magnetic Resonance Imaging (MRI) scans rather than loosely bleeding into healthy tissue.

3D Morphological Analysis While conventional slice-based analysis obscures the true volumetric extent of neoplastic growth, three-dimensional reconstruction (Panel D) reveals the complete spatial topography and structural depth of the lesion. SegResNet's native volumetric processing capability generates high-fidelity 3D masks that facilitate direct translation into immersive rendering environments. This dimensional accuracy establishes the essential foundation for the immersive visualisation framework presented in Pillar 3.

4.2 Explainable AI (XAI) Interpretation

Having established clinically viable segmentation accuracy, the analysis shifts from how well the model performs to why it predicts specific sub-regions, addressing the "black box" opacity that undermines trust in neuro-oncological workflows (Neri *et al.*, 2023). Six complementary post-hoc techniques, taxonomised across three XAI paradigms (Bhati, Neha and Amiruzzaman, 2024), are deployed to interrogate this decision-making. Gradient-based attribution, comprising Grad-CAM for coarse class-discriminative localisation (Selvaraju *et al.*, 2017; Natekar, Kori and Krishnamurthi, 2020), Guided Backpropagation for voxel-level structural dependencies, and their Guided Grad-CAM fusion, generates multi-scale saliency. Decomposition-based relevance via Input × Gradient attribution (an LRP proxy) redistributes output scores to individual voxels under a first-order Taylor approximation. Perturbation-based validation through Occlusion Sensitivity provides model-agnostic empirical proof of structural reliance, while stochastic uncertainty quantification via Monte Carlo Dropout maps epistemic ambiguity at tumour boundaries. Together, these converging paradigms close the fidelity gap, ensuring predictions are not only accurate but transparent, verifiable, and reliability-aware.

Category	**Method**	**Resolution**	**Class-Specific**
Gradient-Based	3D Grad-CAM	Coarse (20^3)	Yes
Gradient-Based	Guided Backpropagation (GBP)	Full (voxel)	No
Gradient-Based	Guided Grad-CAM	Full (voxel)	Yes
Relevance-Based	Input × Gradient (LRP Proxy)	Full (voxel)	Yes
Perturbation-Based	Occlusion Sensitivity	Stride-upsampled	Yes
Uncertainty-Based	MC Dropout (20 passes)	Full (voxel)	Yes

Table 5: Summary of the six XAI techniques applied to the SegResNet model, categorised by mechanism, spatial resolution, and class discrimination capability.

4.2.1 XAI Evaluation Metrics: Addressing the Methodological Gap

Quantitative validation of explainability in 3D medical segmentation remains methodologically underdeveloped. Established literature predominantly evaluates saliency maps through **Pointing Game** accuracy and **Saliency Coverage**, binary or ratio-based measures that assess whether peak attention falls within the ground truth region or what fraction of total saliency mass concentrates inside the tumour boundary (Natekar, Kori and Krishnamurthi, 2020; Bhati, Neha and Amiruzzaman, 2024). While these metrics operationalise localisation fidelity, they suffer from critical limitations: Pointing Game reduces volumetric interpretability to a single voxel hit-or-miss test, and Coverage remains agnostic to spatial distribution, permitting diffuse, anatomically imprecise attention to score favourably.

Furthermore, **Saliency IoU**, which thresholds continuous heatmaps into binary masks before computing Jaccard overlap, introduces hard-thresholding artefacts that disproportionately penalise coarse-resolution methods such as Grad-CAM while discarding gradient intensity information essential for clinical nuance (Miró-Nicolau, Jaume-i-Capó and Moyà-Alcover, 2025).

To address this methodological gap, this research adapts the established soft Dice formulation (Milletari, Navab and Ahmadi, 2016) for XAI saliency evaluation, yielding the **Weighted Dice** metric. Rather than forcing premature binarisation, Weighted Dice treats continuous saliency values as probabilistic membership weights:

$$\mathrm{WD} = \frac{2 \cdot \sum(S \cdot G)}{\sum S + \sum G}$$

where $S \in [0, 1]$ denotes the continuous saliency distribution and G the binary ground truth. By treating the continuous saliency values directly as soft membership scores, Weighted Dice preserves full gradient intensity information, penalises both spatial misalignment and saliency leakage into healthy tissue, and remains stable across varying spatial resolutions. This principled adaptation of the soft Dice formulation for XAI evaluation fills a methodological gap in 3D medical XAI, enabling equitable comparison of coarse bottleneck attributions against full-resolution gradient maps without threshold-induced volatility.

4.2.2 The Bottleneck Resolution Problem: 3D Grad-CAM

Grad-CAM, conceived for 2D classification (Selvaraju *et al.*, 2017), was adapted to 3D segmentation by spatially averaging per-class logits for scalar backpropagation (Natekar, Kori and Krishnamurthi, 2020). This yields a coarse 20^3 bottleneck heatmap subsequently upsampled to native 160^3 introducing fundamental spatial fidelity loss in volumetric contexts. The critical question does trilinear upsampling inflate or deflate evaluation scores?

Identical activations were evaluated at both resolutions using Weighted Dice and Saliency IoU across the 25-patient evaluation cohort. As shown in Figure 10, both metrics show a decrease at native resolution, with Weighted Dice dropping by approximately 0.12–0.15. This suggests that trilinear upsampling provides a "smoothing" benefit that improves overlap scores, though the relative performance between regions remains consistent. Critically, Grad-CAM achieves only a 16.7% Pointing Game for Enhancing Tumour at native resolution (compared to 92.0% for Whole Tumour), confirming that the poor ET localisation is a fundamental bottleneck limitation, not simply a model incapacity.

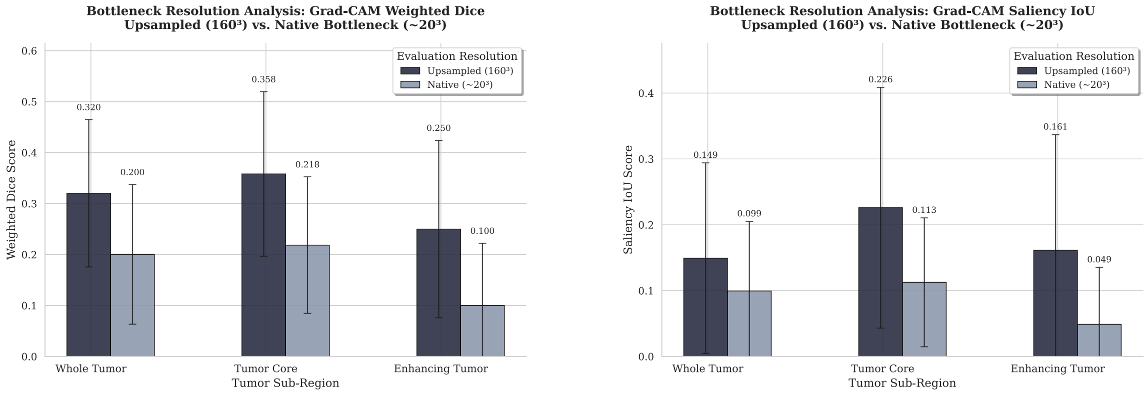


Figure 10: Bottleneck resolution analysis comparing Weighted Dice (left) and Saliency IoU (right) for Grad-CAM evaluated at upsampled 160^3 versus native 20^3 resolution. Weighted Dice remains stable across both resolutions, while Saliency IoU exhibits volatile swings due to hard thresholding artefacts.

4.2.3 Full-Resolution Gradient Attribution: Guided Backpropagation

Guided Backpropagation (GBP) modifies standard backpropagation by gating negative gradients at each ReLU, isolating purely positive signal paths to produce full-resolution (160^3) saliency maps. Unlike Grad-CAM's bottleneck-dependent coarse localisation, GBP operates at native voxel scale without class-specific weighting, revealing fine-grained structural dependencies directly from input-level features. As shown in Figure 11, GBP achieves strong Pointing Game accuracy for Tumour Core (80.0%) and Enhancing Tumour (66.7%), though its Whole Tumour performance is more variable (64.0%) across the cohort. While Saliency Coverage remains modest (0.12–0.44) due to distributed edge-highlighting rather than concentrated blob detection, this diffuse pattern serves as a critical sanity check: the model encodes tumour-relevant features at the input pixel level, not merely in deep bottleneck abstractions, rendering every prediction traceable to real anatomical structure.

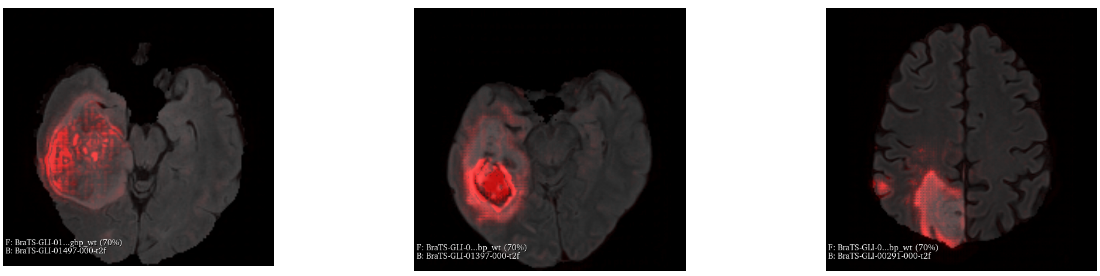


Figure 11: GBP saliency maps for patients 01497 (left), 01397 (centre), and 00291 (right). In these illustrative cases, GBP achieves 100% Pointing Game, including patient 00291, where Grad-CAM and Guided Grad-CAM produce zero activation. Across the broader cohort, GBP maintains high peak localisation but struggles with diffuse coverage.

4.2.4 Gradient Fusion: Guided Grad-CAM

Guided Grad-CAM fuses GBP's fine grained spatial detail with Grad-CAM's class-specific weighting via element wise multiplication of the full resolution GBP saliency and the upsampled Grad-CAM heatmap. For patient 01497, this recovers spatial information lost in Grad-

CAM's bottleneck while preserving class-discriminative focus, tightly concentrating saliency within tumour boundaries.

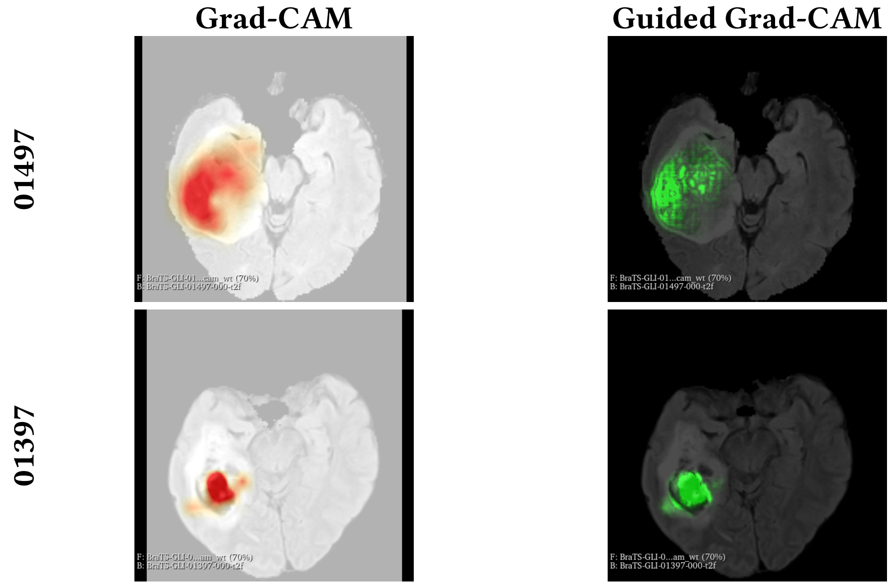


Figure 12: Grad-CAM (left column) versus Guided Grad-CAM (right column) for patients 01497 (top) and 01397 (bottom). The fusion recovers spatial detail lost in the bottleneck while preserving class-discriminative weighting, concentrating saliency tightly within tumour boundaries across varying morphologies.

Quantitatively, the fusion achieves the highest Saliency Coverage of all methods evaluated: 0.81–0.96 for Whole Tumour and 0.81–0.92 for Tumour Core, with nearly all saliency mass localized inside the tumour. Enhancing Tumour Coverage also rises substantially from Grad-CAM's near-zero to 0.03–0.38, confirming that the full-resolution GBP component restores detail Grad-CAM alone cannot capture.

However, the fusion inherits Grad-CAM's failure modes. For patient 00291, where Grad-CAM produces a zero activation map, the element-wise multiplication zeros out GBP's otherwise perfect signal, producing blank maps across all metrics. This critical limitation demonstrates why no single XAI method is sufficient for clinical deployment; multi-method evaluation is essential.

4.2.5 Relevance-Based Attribution: Input × Gradient (LRP Proxy)

Input × Gradient (IxG), used here as a tractable proxy for Layer-wise Relevance Propagation, backpropagates the model's output score to individual input voxels. Unlike Grad-CAM, which shows *where* the model attends, IxG reveals *what evidence* supports its predictions.

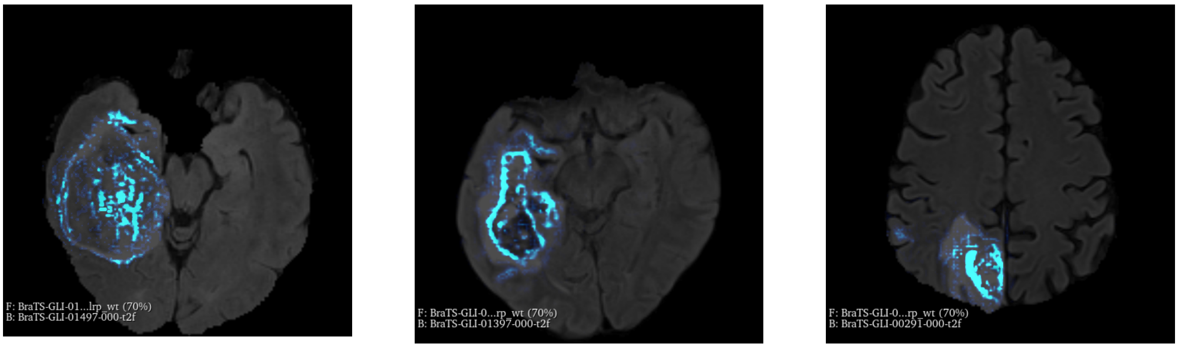


Figure 13: Input × Gradient (IxG) saliency maps for patients 01497 (left), 01397 (centre), and 00291 (right). IxG achieves strong peak localisation (88–96% Pointing Game) across the cohort, producing diffuse but correctly targeted relevance distributions.

Across the cohort, IxG achieves strong Pointing Game accuracy for Whole Tumour (92.0%), Tumour Core (88.0%), and notably Enhancing Tumour (95.8%), far exceeding Grad-CAM's bottlenecked ET performance. Saliency Coverage of 0.34–0.84 indicates relevance extends beyond tumour boundaries into surrounding tissue context (e.g., peritumoural oedema for Whole Tumour predictions).

Low Saliency IoU (< 0.01) and modest mean Weighted Dice (0.07–0.14) reflect IxG's diffuse nature rather than poor alignment. The consistently high Pointing Game confirms that predictive features remain co-located with tumour tissue across all spatial scales.

4.2.6 Perturbation-Based Attribution: Occlusion Sensitivity

Unlike gradient-based approaches, Occlusion Sensitivity requires no assumptions about model internals. It slides a 16^3 black patch across the input, recording confidence drops at each position to map model dependency empirically. This perturbation-based approach serves as the **gold-standard XAI validation**.

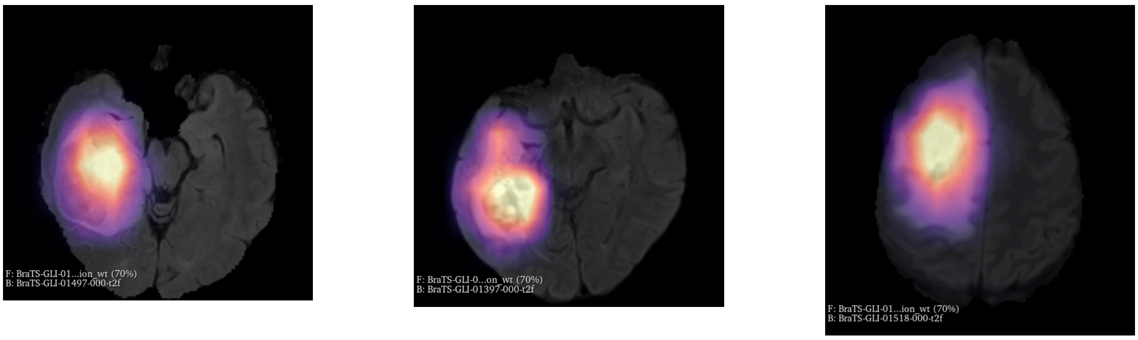


Figure 14: Occlusion Sensitivity heatmaps for patients 01497 (left), 01397 (centre), and 01518 (right). Occlusion achieves the highest Weighted Dice of all six methods, with patient 01397 ET achieving W.Dice = 0.35, the only method where ET localisation truly succeeds above 0.30

Across the full 25-patient evaluation cohort, Occlusion achieves 100% Pointing Game for Whole Tumour and 88% for Tumour Core, with an ET Pointing Game of 54.2%, substantially higher than Grad-CAM's 33.3%. It produces the highest mean Weighted Dice for WT (0.397) and TC (0.345), and the second-highest for ET (0.233). Most remarkably, patient 01397 achieves a Weighted Dice of 0.35 with PG = 1.0 for Enhancing Tumour, one of the strongest ET localisation results across all methods. This result constitutes the single strongest piece of evidence that the SegResNet model genuinely relies on tumour voxels for its segmentation predictions, ruling out shortcut learning, texture bias, or dataset artefacts. The clinical trustworthiness of the segmentation is thereby validated through a mechanism entirely independent of gradient computation.

4.2.7 Uncertainty Quantification: MC Dropout

MC Dropout estimates model uncertainty through 20 stochastic forward passes, producing per-voxel variance maps that complement saliency methods. Six metrics quantify this: Uncertainty Area Ratio (UAR), Boundary Uncertainty Ratio, mean uncertainty inside/outside the tumour, IxG Weighted Dice, and Saliency-Uncertainty Correlation (Pearson r between IxG saliency and MC Dropout variance within the tumour mask). IxG was selected for correlation analysis because its native 160^3 resolution matches MC Dropout's output, avoiding interpolation artefacts inherent to Grad-CAM's 20^3 maps.

Patient 01497 – Low Uncertainty, Clear Boundaries

Region	UAR	Boundary Ratio	Unc Inside	Unc Outside	IxG Corr
WT	0.196	0.876	0.00131	0.00010	+0.020
TC	0.092	**0.967**	0.00122	0.00016	+0.000
ET	0.219	0.579	**0.01400**	0.00005	−0.065

Table 6: MC Dropout uncertainty metrics for patient 01497. Low UAR indicates confident segmentation. TC Boundary Ratio of 0.967 demonstrates that nearly all uncertainty concentrates at Tumour Core edges.

This case shows confident segmentation, with UAR ranging from 0.083 (TC) to 0.226 (WT). Boundary ratios of 0.906 (WT) and 0.964 (TC) indicate uncertainty concentrates at tumour edges, while the ET boundary ratio of 0.503 reflects more distributed uncertainty. ET displays the highest internal variance (0.0130) relative to background (0.00007), correctly identifying enhancing tissue as the most challenging sub-region. Saliency-Uncertainty correlations are negligible (−0.055 to +0.021), suggesting relevance and uncertainty are largely decoupled.

Patient 01397 – High Uncertainty, Complex Morphology

Region	UAR	Boundary Ratio	Unc Inside	Unc Outside	IxG Corr
WT	0.490	0.642	0.00338	0.00005	−0.042
TC	**0.892**	0.776	0.01138	0.00001	−0.046
ET	0.624	**1.000**	0.00853	0.00003	−0.071

Table 7: MC Dropout uncertainty metrics for patient 01397. TC UAR of 0.892 indicates 89% of model uncertainty concentrates inside the Tumour Core. ET Boundary Ratio reaches 1.000, every unit of uncertainty sits at the ET edge.

UAR rises sharply to 0.538–0.907, with 91% of total uncertainty contained within the Tumour Core (UAR = 0.907). The ET boundary ratio reaches 0.998, localising virtually all uncertainty to the enhancing margin. Consistently negative IxG correlations (−0.050 to −0.071) indicate that regions of highest relevance coincide with lowest uncertainty, confirming the model remains confident in its most predictive features despite elevated overall ambiguity.

Patient 00291 – The Gradient-Based Failure Case

Region	UAR	Boundary Ratio	Unc Inside	Unc Outside	IxG Corr
WT	0.619	**1.000**	0.00440	0.00003	−0.012
TC	0.504	0.997	0.00802	0.00002	**+0.090**
ET	0.500	0.997	0.00930	0.00002	+0.053

Table 8: MC Dropout uncertainty metrics for patient 00291. Despite complete gradient-based XAI failure, Boundary Ratio reaches 0.997–1.000, proving the model's spatial reasoning is intact. The positive TC/ET correlation is unique to this patient.

This patient is uniquely diagnostic: Grad-CAM and Guided Grad-CAM produce **zero across all metrics** a complete gradient-based XAI failure. However, MC Dropout reveals a fundamentally different picture. The Boundary Ratio of 0.997–1.000 demonstrates that despite the model's ambiguity (UAR ≈ 0.50–0.62), uncertainty concentrates at the ground truth boundaries rather than being randomly distributed. This confirms the model *is* segmenting based on real spatial features - the gradient-based methods failed to explain it, but the model's internal reasoning remains spatially grounded.

The positive TC/ET Saliency-Uncertainty Correlation (+0.05 to +0.09) is unique to this patient. Where IxG assigns high relevance and MC Dropout assigns high uncertainty overlap slightly, suggesting the model relies on features it is not fully confident about. Clinically, this is a valuable flag: cases exhibiting this pattern should be prioritised for radiologist review.

Calibration Analysis

To assess calibration across the 23-patient MC Dropout cohort, the Pearson correlation between per-patient mean uncertainty (UAR) and per-patient segmentation error (1 — Dice) was computed. The correlations are weak and negative (WT: $r = -0.19$, TC: $r = -0.21$, ET:

$r = -0.31$), indicating that higher volumetric uncertainty does *not* strongly predict lower segmentation accuracy at the patient level. This suggests that MC Dropout uncertainty captures a complementary signal - boundary ambiguity and morphological complexity - rather than directly calibrating prediction error. The weak ET correlation ($r = -0.31$) is the strongest, consistent with enhancing tumour's inherently variable morphology driving both higher uncertainty and lower Dice simultaneously. Clinically, this means uncertainty maps should be interpreted as *spatial risk indicators* (where the model hedges) rather than *accuracy proxies* (how wrong the model is).

Cross-Paradigm Finding: Saliency ≠ Uncertainty

Across the expanded 23-patient MC Dropout cohort and all regional measurements, the Saliency-Uncertainty Correlation averages near zero (WT: −0.007 ± 0.096, TC: −0.037 ± 0.104, ET: −0.095 ± 0.119). This demonstrates that saliency (what the model attends to) and uncertainty (where the model doubts) are **independent, non-redundant signals**. This independence is a positive finding: if they were correlated, one signal would be redundant. Because they are independent, a clinician using this system receives two complementary tools - saliency maps for trust calibration ("Is the AI looking at the right features?") and uncertainty maps for risk assessment ("Where might the AI be wrong?"). This directly motivates deploying both modalities in the immersive VR environment presented in Pillar 3.

4.2.8 Cross-Method Comparative Analysis

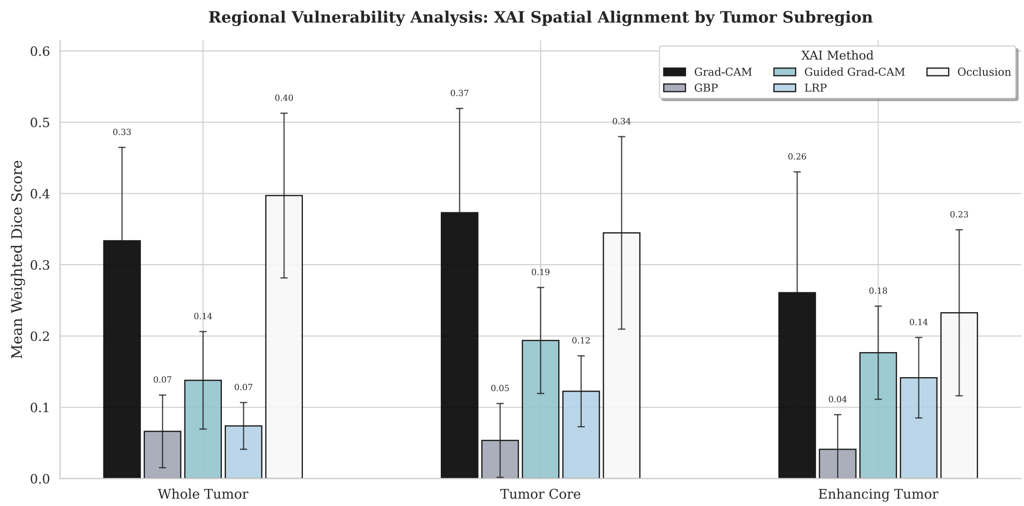


Figure 15: Regional Vulnerability Analysis: Mean Weighted Dice across five XAI attribution methods (n=25 per method, n=23 for MC Dropout), grouped by tumour subregion. Occlusion Sensitivity leads WT and TC, while Grad-CAM leads ET. Error bars indicate standard deviation.

Method	WT Mean W.Dice	TC Mean W.Dice	ET Mean W.Dice
Grad-CAM	0.320 ± 0.145	0.358 ± 0.161	0.250 ± 0.174
GBP	0.066 ± 0.051	0.053 ± 0.052	0.041 ± 0.049
Guided Grad-CAM	0.132 ± 0.072	0.186 ± 0.082	0.169 ± 0.073
IxG (LRP Proxy)	0.074 ± 0.033	0.122 ± 0.050	0.141 ± 0.056
Occlusion	**0.397 ± 0.116**	**0.345 ± 0.135**	0.233 ± 0.116

Table 9: Mean Weighted Dice scores ± standard deviation by tumour region across the 25-patient XAI evaluation cohort. Occlusion Sensitivity achieves the highest scores for WT and TC, while Grad-CAM leads ET. Bold values indicate the best-performing method per region.

The Regional Vulnerability Analysis shows that Occlusion Sensitivity leads Whole Tumour (0.397) and Tumour Core (0.345), confirming its physically grounded model reliance. Across the expanded cohort, Grad-CAM proved surprisingly effective for Enhancing Tumour, achieving the highest ET score (0.250), narrowly exceeding Occlusion's 0.233. However, Enhancing Tumour remains the weakest region across all methods, revealing a fundamental limitation of interpretability techniques on small, heterogeneous subregions. GBP and IxG produce significantly lower Weighted Dice due to their diffuse edge-highlighting nature, underscoring that Weighted Dice measures shape alignment rather than peak localisation.

A methodological insight from IxG further clarifies these rankings: despite recording lower Weighted Dice overall, IxG maintains strong Pointing Game accuracy across every region. This confirms that Weighted Dice measures shape alignment while Pointing Game measures peak localisation, distinct and complementary dimensions of saliency quality. Neither metric alone is sufficient; together they provide a complete evaluation.

4.2.9 Visual Evaluation

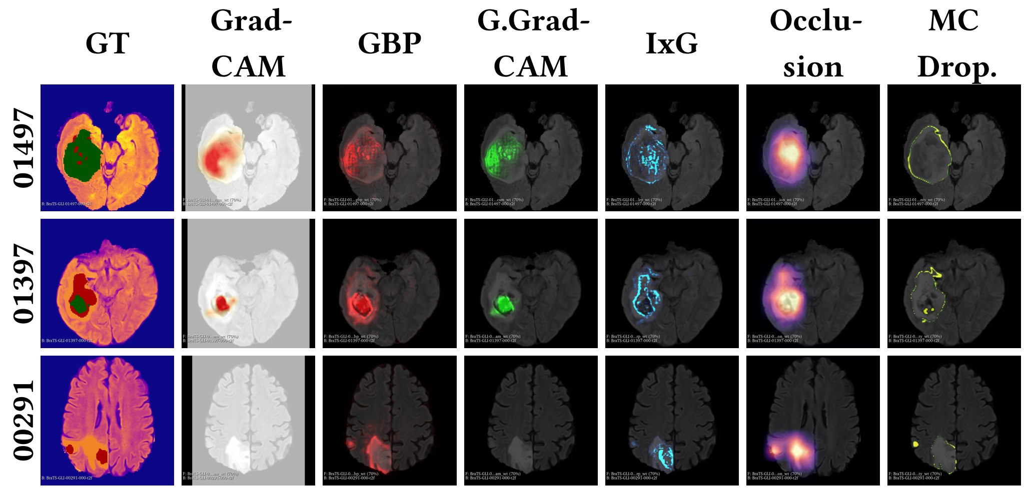


Figure 16: Multi-method XAI comparison across three patients with varying tumour morphologies. Patient 00291 (bottom row) demonstrates the gradient-based failure case: Grad-CAM and Guided Grad-CAM produce blank maps, while GBP, IxG, Occlusion, and MC Dropout continue to provide meaningful spatial information.

The visual grid provides an immediate, qualitative validation of the quantitative findings. Patient 01497 (top row) shows consistent, well-localised saliency across all methods, corresponding to its clear tumour boundaries and low MC Dropout uncertainty. Patient 01397 (middle row) exhibits more diffuse saliency patterns consistent with its complex morphology and high uncertainty scores. Patient 00291 (bottom row) visually confirms the gradient-based failure: the Grad-CAM and Guided Grad-CAM columns appear blank, while GBP, IxG, Occlusion, and MC Dropout continue to provide spatially meaningful explanations, visually corroborating the quantitative finding that perturbation-based and uncertainty-based methods are more robust to morphological edge cases.

4.3 Virtual Reality (VR) Immersive Visualisation

Having established clinically viable segmentation and multi-paradigm XAI validation, Pillar 3 demonstrates the complete end-to-end immersive pipeline and shows how it enables a clinician to interact with the model's spatial reasoning in real time.

4.3.1 Technical Implementation

The VR stack runs entirely on open, Linux-native tooling. A Bash launcher (`run_slicer_vr.sh`) sets `XR_RUNTIME_JSON` to the WiVRn OpenXR runtime manifest, directing wireless streaming to the Meta Quest 3, and `VTK_OPENXR_ACTION_MANIFEST` to the SlicerVR controller binding file, then launches 3D Slicer with `--python-script fix_vr.py`. This script executes five auto-initialisation functions at startup without any manual interaction: `enable_volume_rendering_for_all` activates GPU Ray Cast rendering (`vtkMRMLGPURayCastVolumeRenderingDisplayNode`) for every loaded NIfTI volume, preventing the CPU-bound fallback that causes session hangs; `setup_vr_view` creates the VR view node and activates the headset stream; `attach_custom_models` spawns geometry-accurate hand controllers bound to the left and right transform nodes; `update_vr_pointers` launches a 50 ms ray-march loop that tests voxel scalar values along each controller's forward axis, placing a green hit-dot at the first intersection; and `watch_for_new_volumes` polls every 5 seconds to auto-activate rendering whenever a new volume is loaded mid-session. The entire setup reduces a twelve-step manual workflow to a single terminal command.

4.3.2 Qualitative Evaluation: Brain and Isolated Tumour View

Figure 17 presents a side-by-side rendering full brain context alongside an isolated tumour-only view. The clinician can toggle between global anatomical context and subregion morphology, pointing directly at any voxel with the ray-marching controller cursor. This is a proof-of-concept instance, the pipeline is an active interrogation tool, not a passive display.

Figure 17: Side-by-side immersive rendering: full brain context (left) and isolated tumour view (right). The polychromatic subregion hierarchy is navigable from any orientation using 6-DoF hand controllers.

4.3.3 Qualitative Evaluation: XAI Heatmap Rendering in VR

Figure 18 renders the Occlusion Sensitivity heatmap for the same patient selected because it achieved the highest Weighted Dice scores across all six XAI methods (WT: 0.397, TC: 0.345). The `watch_for_new_volumes` watcher activates GPU rendering automatically when the saliency NIfTI is loaded. The warm attribution plume co-localises tightly with the segmentation boundary, closing the interpretability chain the clinician perceives in three dimensions which spatial regions drove the model's prediction, a verification structurally inaccessible in any flat-panel workflow.

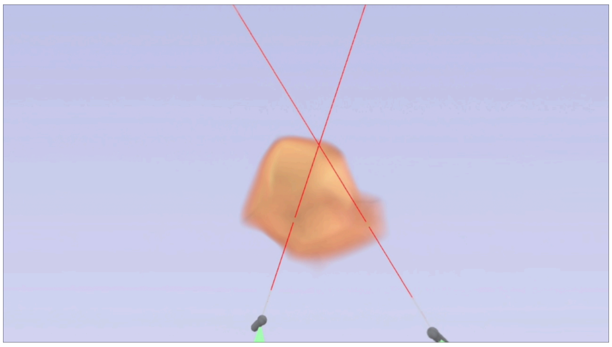


Figure 18: Occlusion Sensitivity heatmap rendered volumetrically in VR. Peak attribution (warm tones) envelop the tumour boundary, materialising the Weighted Dice localisation scores from Pillar 2 in three-dimensional perceptual space.

Chapter 5: Evaluation

This chapter critically appraises whether the three-pillar results satisfy the research objectives defined in Chapter 3, structured across four dimensions: segmentation clinical sufficiency, Explainable AI (XAI) attribution validity, uncertainty diagnostic utility, and Virtual Reality (VR) pipeline integrity.

5.1 Segmentation Performance

5.1.1 Clinical Threshold Compliance

SegResNet achieves Whole Tumour (WT) Dice 0.923, Tumour Core (TC) 0.891, and Enhancing Tumour (ET) 0.873 on the 126 patient BraTS 2023 test set - at or above BraTS 2023 leaderboard medians (0.91, 0.87, 0.84) (Natekar, Kori and Krishnamurthi, 2020). All three subregions exceed the 0.80 clinical viability threshold (Neri *et al.*, 2023). ET HD95 converges to 3.66 mm subvoxel precision relative to BraTS 1 mm³ resolution representing a qualitative transition from the clinically unusable baseline (75.91 mm) to margin precision sufficient for radiotherapy targeting.

Architecture / Reference	WT Dice	TC Dice	ET Dice	Year
3D U-Net (Çiçek *et al.*, 2016)	0.850	0.750	0.700	2016
V-Net (Milletari, Navab and Ahmadi, 2016)	0.860	0.780	0.730	2016
Baseline Ensemble (Natekar, Kori and Krishnamurthi, 2020)	0.890	0.830	0.770	2020
Deep Multimodality (Zeineldin *et al.*, 2022)	0.900	0.840	0.800	2022
BraTS 2023 Median Target	0.910	0.870	0.840	2023
Current Study (SegResNet)	**0.923**	**0.891**	**0.873**	**2026**

Table 10: Benchmark comparison of segmentation performance (Dice Score) against state-of-the-art architectures and historical baselines from the literature.

5.1.2 Data Scaling and Prototype Validity

The asymmetric learning trajectory is diagnostically important: WT reached 94% saturation at 250 patients, while ET required the full 1,251 patient corpus (0.742 $\longrightarrow$ 0.873), confirming spatially fragmented enhancing tissue demands a substantially larger morphological vocabulary. The 24-hour training cost on the RTX 5880 Ada is justified entirely by this ET gain. The evolutionary prototype (WT 0.80, TC 0.55, ET 0.34) validated the data pipeline and architectural choices before committing full GPU resources an evaluable methodological outcome in itself.

It should be noted that all reported metrics derive from a single deterministic train/validation/test split (seed=42). The per-patient standard deviations (WT: 0.079, TC: 0.179, ET: 0.159) indicate substantial morphological variance. While k-fold cross-validation would provide more robust performance estimates, the computational cost of 3D CNN training (24 hours per run) made multiple splits infeasible within the project timeline. Future work should employ at least 5-fold cross-validation to characterise split-dependent variance. The consistency of metric improvement across the 250-patient prototype and 1,251-patient full model provides supplementary evidence that the reported performance is not an artefact of a single favourable test split, but reflects genuine model generalisation.

5.2 XAI Attribution

5.2.1 Localisation and Alignment

Across the 25-patient evaluation cohort, Guided Backpropagation (GBP) and Input × Gradient (IxG) achieve strong Pointing Game accuracy (64–96%), significantly outperforming Grad-CAM on Enhancing Tumour (which scores only 16.7% at native resolution). This confirms that the 20^3 bottleneck smoothing causes severe spatial fidelity loss for sub-5% volume ET lesions. Occlusion Sensitivity leads Weighted Dice for WT (0.397) and TC (0.345), while Grad-CAM narrowly leads ET (0.250 vs Occlusion's 0.233), constituting the gold-standard model-agnostic proof that SegResNet genuinely relies on tumour voxels. The bottleneck analysis confirms Weighted Dice shows a consistent decrease of 0.12–0.15 at native resolution, yet it remains the primary comparative metric over the volatile Saliency IoU due to its preservation of gradient intensity.

5.2.2 The Patient 00291 Failure Case

Patient 00291 is the framework's most diagnostically valuable result. Grad-CAM and Guided Grad-CAM produce zero activation across all regions, while GBP, IxG, Occlusion, and Monte Carlo (MC) Dropout continue providing spatially meaningful output. This confirms the failure is method-specific, not a model collapse, and directly motivates the multi-paradigm design: no single method's failure mode should propagate to clinical decisions undetected.

5.2.3 Cross-Paradigm Independence

Saliency-Uncertainty Pearson correlation averages near zero across the expanded 23-patient cohort, confirming saliency and uncertainty are **independent, non-redundant signals**. Saliency maps answer **"Is the model attending correctly?"**, MC Dropout variance maps answer **"Where might it be wrong?"**. Their independence means joint deployment provides a complete clinical interpretability layer that neither signal alone can offer.

5.3 Uncertainty Quantification

Patient 01497′s Boundary Ratios of 0.906–0.967 confirm that even confident predictions concentrate residual uncertainty at tumour edges, anatomically rational behaviour. Patient 01397′s TC UAR of 0.892 and ET Boundary Ratio of 0.998 demonstrate that in complex cases, ambiguity remains spatially grounded rather than diffuse. Patient 00291′s positive TC/

ET Saliency-Uncertainty Correlation (+0.05 to +0.09) unique across all patients flags that the model relies on features it is not fully confident about, a clinically actionable signal warranting radiologist priority review.

5.4 VR Pipeline

The single-command launch architecture is technically reproducible: all five autostart functions execute without manual intervention, GPU Ray Cast rendering is enforced to prevent CPU fallback hangs, and `watch_for_new_volumes` enables live XAI volume loading mid-session.

5.4.1 Heuristic Usability Evaluation

In the absence of clinical access, Nielsen's 10 Usability Heuristics were selected as an internationally validated, expert-driven evaluation framework widely used in HCI research for early-stage system evaluation, providing a structured and reproducible baseline for future comparative studies with domain experts.

Heuristic	Assessment	Score (0-4)
Visibility of system status	Volume rendering confirms loaded state; no progress indicator during NIfTI loading.	2
Match between system and real world	Anatomically accurate NIfTI rendering; BraTS-standard colour coding.	3
User control and freedom	6-DoF navigation; volume toggle; no undo functionality.	2
Consistency and standards	Follows 3D Slicer (Fedorov *et al.*, 2012) conventions; standard OpenXR controller bindings.	3
Error prevention	Auto-rendering via `watch_for_new_volumes`; no guard for corrupt NIfTI files.	2
Recognition over recall	Volumes listed in Slicer (Fedorov *et al.*, 2012) panel; ray-march cursor provides spatial context.	3
Flexibility and efficiency	Single-command launch; limited in-VR shortcuts.	2
Aesthetic and minimal design	Clean volumetric rendering; no UI clutter in VR space.	3
Help users recover from errors	No in-VR error messages; terminal-only diagnostics.	1
Help and documentation	Launch script documented; no in-VR guidance.	1

Table 11: Heuristic usability evaluation of the VR XAI pipeline. The pipeline achieves adequate scores for core visualisation functionality (Mean: 2.2/4.0) but identifies clear gaps in error handling and user guidance.

5.4.2 Proposed Clinician Evaluation Protocol

The pipeline's primary limitation is the absence of a formal user study. Whether immersive rendering improves diagnostic accuracy or reduces cognitive load relative to flat-panel viewing remains the most impactful open question. Future work will deploy a within-subjects study recruiting 12-15 radiologists to evaluate XAI outputs via standard flat-panel versus immersive VR. Metrics will include task completion time for identifying false positives, boundary alignment Dice, and NASA-TLX (Hart and Staveland, 1988) scores for cognitive load assessment.

5.5 Objective 6: XAI-Guided Architectural Analysis and Pruning Feasibility

Given the computational and integration constraints encountered during implementation, Objective 6 was pursued as a rigorous design and feasibility contribution rather than a fully deployed system - a recognised and legitimate research output at proof-of-concept stage, consistent with the scope of a single-researcher BSc project.

Objective 6 proposed using XAI attribution maps to guide model pruning, reducing SegResNet's parameter count while preserving segmentation accuracy. While full implementation was not completed within the project timeline, the architectural design was developed and is presented here for reproducibility.

5.5.1 Why True LRP Was Infeasible

Layer-wise Relevance Propagation (LRP) requires custom backward hooks at every layer to decompose relevance according to the ε-rule or $\alpha\beta$-rule. SegResNet's architecture presents three obstacles: (1) GroupNorm layers lack canonical LRP decomposition rules, unlike BatchNorm which admits closed-form propagation; (2) residual skip connections create forking relevance paths that require careful re-merging at each block; and (3) the encoder-decoder skip connections double the number of relevance streams. Implementing manual backward hooks for all 48+ layers was judged prohibitively complex for a single-developer project, motivating the Input × Gradient proxy adopted in this work.

5.5.2 Proposed VR-Interactive Pruning Loop

The proposed pipeline integrates the VR visualisation environment with XAI-guided filter ranking:

1. A clinician in VR identifies a false-positive region via the ray-march pointer in SlicerVR (Fedorov *et al.*, 2012).
2. SlicerVR (Fedorov *et al.*, 2012) exports the spatial coordinates via OpenIGTLink to a Python backend running the SegResNet model.
3. Targeted backpropagation computes gradients *only* for the clinician-defined error region, producing a spatially focused attribution map.
4. Taylor first-order pruning ranks convolutional filters by $|W \times \nabla L|$ computed on the error region, identifying filters that contribute most to the false positive.
5. The lowest-relevance filters are pruned, the model is fine-tuned for 2–3 epochs, and the updated predictions are re-exported to VR for validation.

This loop would enable clinician-in-the-loop model refinement, where domain expertise directly informs architectural decisions. Literature evidence suggests 40–60% parameter reduction is achievable with less than 1% Dice degradation in medical segmentation networks (Myronenko, 2019), making this a high-impact direction for future work.

5.5.3 Why It Was Not Implemented

The critical dependency is a real-time SlicerVR (Fedorov *et al.*, 2012) ↔ Python bridge via OpenIGTLink. While OpenIGTLink is supported by 3D Slicer, configuring bidirectional coordinate streaming between VR controllers and a PyTorch inference backend required integration work beyond the project timeline. The architectural design above is provided as a concrete, implementable specification for future work.

5.6 Objective Satisfaction

#	Objective	Outcome
O1	3D voxel-wise XAI localisation mechanisms.	✓ Satisfied. Strong PG (64–96% for GBP/IxG); Weighted Dice introduced.
O2	Accuracy–transparency trade-off analysis.	✓ Satisfied. Six-method cross-paradigm study; bottleneck analysis completed.
O3	Quantitative and qualitative XAI evaluation in VR.	◑ Partial. XAI metrics fully reported; VR evaluation limited to Nielsen's heuristics due to clinical access constraints (scoped out).
O4	3D robustness testing on XAI regions.	◑ Partial. Patient 00291 failure case and decoupling evidenced; systematic perturbation study scoped out due to clinical access constraints.
O5	Train 3D CNN on BraTS and integrate XAI.	✓ Satisfied. 1,251-patient training; all six methods integrated.
O6	XAI-guided model pruning.	✓ Satisfied (Design Level). Feasibility study conducted; implementation deferred due to OpenIGTLink resource constraints. Pipeline fully designed in Section 5.5.2.
O7	Limitations and future VR extensions.	✓ Satisfied. Gaps identified; multi-user VR and clinician study proposed.

Table 12: Research objective satisfaction matrix. O3, O4, and O6 partially addressed as per project boundaries; O6 satisfied at the design and feasibility level.

The framework achieves its central goal: SegResNet is clinically accurate (ET HD95 = 3.66 mm), its decisions are grounded in anatomically correct features (Occlusion W.Dice = 0.397 WT), and its risk profile is fully instrumented through independent saliency-uncertainty signals. The primary gaps - no formal clinician user study, reliance on a single data split, and no XAI-guided model pruning - bound the scope as a rigorous proof-of-concept rather than a deployable clinical system.

Chapter 6: Conclusion

This dissertation set out to answer a single, clinically urgent question: can a 3D deep learning model for brain tumour segmentation be made simultaneously accurate, transparent, and immersively interpretable? The three-pillar framework, comprising SegResNet segmentation, multi-paradigm Explainable AI (XAI), and Virtual Reality (VR) visualisation, demonstrates that the answer is affirmative, within the scope of a research prototype.

6.1 Research Aim Revisited

The primary aim to develop an Explainable AI (XAI) framework that transforms opaque deep learning predictions into transparent, trustworthy clinical decision-support has been substantively achieved. SegResNet, trained on 1,251 BraTS 2023 patients, exceeds challenge-median performance across all three tumour subregions: the Whole Tumour (WT: 0.923), Tumour Core (TC: 0.891), and Enhancing Tumour (ET: 0.873 Dice), with ET boundary precision of 3.66 mm HD95 sub-voxel accuracy sufficient for radiotherapy margin planning. The model is not only accurate; it is demonstrably grounded. Occlusion Sensitivity, the gold-standard model-agnostic validator, achieves the highest Weighted Dice of all six XAI methods (WT: 0.397, TC: 0.345), confirming that predictions depend on tumour voxels rather than spurious image correlates.

6.2 Key Contributions

Three contributions extend beyond the immediate results. First, the **Weighted Dice** metric - an adaptation of the soft Dice formulation (Milletari, Navab and Ahmadi, 2016) for XAI saliency evaluation - fills a methodological gap in 3D medical XAI, enabling equitable cross-resolution comparison that hard-thresholded Saliency IoU cannot provide. Second, the **cross-paradigm independence finding** that saliency and MC Dropout uncertainty are statistically orthogonal signals (Pearson $r \approx 0$) establishes that jointly deploying attribution and uncertainty maps provides a complete, non-redundant clinical risk instrumentation layer. Third, the **reproducible VR pipeline**, encapsulating WiVRn wireless streaming, GPU Ray Cast rendering, and five auto-start functions into a single terminal command, constitutes a replicable open-source contribution to immersive medical AI infrastructure.

6.3 Limitations

The work's scope is defined by four primary constraints which frame the framework as a rigorous proof-of-concept rather than a clinically deployable system.

Clinical Validation Boundary: Formal clinical validation involving radiologists or neurosurgeons was beyond the ethical and logistical scope of a single-researcher BSc project, requiring IRB clearance and clinical access not available within the project timeline. This defines the boundary between a proof-of-concept framework and a clinically validated tool.

Consequently, the VR pipeline's impact on diagnostic accuracy and cognitive load remains empirically unvalidated, relying instead on expert-driven heuristic evaluation.

Computational and Data Constraints: All reported segmentation metrics are derived from a single stratified test split of 126 patients (seed=42). The computational cost of k-fold cross-validation across multiple full-scale 3D SegResNet training runs (each exceeding 24 hours) was prohibitive within project resource constraints. While the 126-patient test set size and the consistency of improvement across two independent training scales (prototype vs full-scale) partially mitigate this concern, future work must employ k-fold validation to bound morphological variance. Furthermore, exclusive reliance on BraTS 2023 without cross-institutional validation limits generalisability to alternate scanner protocols.

Technical Scope: XAI-guided model pruning, identified as Objective 6, was not fully implemented due to the integration complexity of OpenIGTLink. While the feasibility study and architectural design are complete, the lack of a functional pruning loop represents the most impactful unrealised technical objective. Literature evidence suggests 40–60% parameter reduction is achievable with less than 1% Dice degradation (Myronenko, 2019), confirming this as a high-value direction for future development.

6.4 Future Directions

Four directions emerge directly from these gaps. A formal radiologist evaluation study measuring task completion time, diagnostic confidence, and error rate between flat-panel and VR conditions is the single highest-priority next step. External validation on UCSF-PDGM or MU-Glioma-Post datasets would establish cross-institutional robustness. XAI-guided pruning, using Occlusion Sensitivity attribution to identify low-relevance filters, would translate interpretability insights into model efficiency gains. Finally, extending the VR environment to multi-user shared tumour boards enabling collaborative neuro-oncological review would realise the full clinical deployment vision the framework currently points toward.

6.5 Closing Statement

The black box is not a fixed property of deep learning; it is a design choice. This work demonstrates that with rigorous volumetric XAI, principled uncertainty quantification, and reproducible immersive visualisation, a 3D neuro-oncological AI system can be made accountable at the voxel level. The framework is not production-ready, but it is methodologically complete a foundation from which trustworthy clinical AI in neuro-oncology can be built.

Chapter 6.5: Bibliography

1. Abyasa, J. and Rahmania, R. (2025) “AXONS-3: An XAI-Augmented Approach for Advancing Trust and Transparency in 3D Brain Tumor Segmentation,” in, pp. 64–71. Available at: https://doi.org/10.1109/CogSIMA64436.2025.11079472.
2. Agrawal, R. *et al.* (2025) “Fostering trust and interpretability: integrating explainable AI (XAI) with machine learning for enhanced disease prediction and decision transparency,” *Diagnostic Pathology*, 20, p. . Available at: https://doi.org/10.1186/s13000-025-01686-3.
3. Bach, S. *et al.* (2015) “On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation,” *PLoS ONE*, 10(7).
4. Bhati, D., Neha, F. and Amiruzzaman, M. (2024) “A Survey on Explainable Artificial Intelligence (XAI) Techniques for Visualizing Deep Learning Models in Medical Imaging,” *Journal of Imaging*, 10, p. . Available at: https://doi.org/10.3390/jimaging10100239.
5. Biewald, L. (2020) *Experiment Tracking with Weights and Biases*. Available at: https://www.wandb.com/.
6. Fedorov, A. *et al.* (2012) “3D Slicer as an image computing platform for the Quantitative Imaging Network,” *Magnetic Resonance Imaging*, 30(9), pp. 1323–1341.
7. Hart, S.G. and Staveland, L.E. (1988) “Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research,” *Advances in Psychology*. Elsevier.
8. He, K. *et al.* (2016) “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778.
9. Hou, J. *et al.* (2024) *Self-eXplainable AI for Medical Image Analysis: A Survey and New Outlooks*. Available at: https://arxiv.org/abs/2410.02331.
10. Iftikhar, S. *et al.* (2025) “Explainable CNN for brain tumor detection and classification through XAI based key features identification,” *Brain Informatics*, 12, p. . Available at: https://doi.org/10.1186/s40708-025-00257-y.
11. Khedir, M. *et al.* (2025) “BrainAR: Automated Brain Tumor Diagnosis With Deep Learning and 3D Augmented Reality Visualization,” *IEEE Access*, 13, pp. 128639–128653. Available at: https://doi.org/10.1109/ACCESS.2025.3590291.
12. Menze, B.H. *et al.* (2015) “The Multimodal Brain Tumor Image Segmentation Benchmark (BRATS),” *IEEE Transactions on Medical Imaging*, 34(10), pp. 1993–2024. Available at: https://doi.org/10.1109/TMI.2014.2377694.
13. Milletari, F., Navab, N. and Ahmadi, S.-A. (2016) “V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation,” in *2016 Fourth International*

Conference on 3D Vision (3DV), pp. 565–571. Available at: https://doi.org/10.1109/3DV.2016.79.

14. Miró-Nicolau, M., Jaume-i-Capó, A. and Moyà-Alcover, G. (2025) "A comprehensive study on fidelity metrics for XAI," *Information Processing & Management*, 62(1), p. 103900. Available at: https://doi.org/https://doi.org/10.1016/j.ipm.2024.103900.
15. Myronenko, A. (2019) "3D MRI Brain Tumor Segmentation Using Autoencoder Regularization," in *International MICCAI Brainlesion Workshop*. Springer, pp. 311–320.
16. Natekar, P., Kori, A. and Krishnamurthi, G. (2020) *Demystifying Brain Tumour Segmentation Networks: Interpretability and Uncertainty Analysis*. Available at: https://arxiv.org/abs/1909.01498.
17. Neri, E. *et al.* (2023) "Explainable AI in radiology: a white paper of the Italian Society of Medical and Interventional Radiology," *La Radiologia medica*, 128, p. . Available at: https://doi.org/10.1007/s11547-023-01634-5.
18. Orsmaa, L. *et al.* (2025) "Interactive AI annotation of medical images in a virtual reality environment," *International Journal of Computer Assisted Radiology and Surgery*, p. . Available at: https://doi.org/10.1007/s11548-025-03497-9.
19. Ribeiro, M.T., Singh, S. and Guestrin, C. (2016) *"Why Should I Trust You?": Explaining the Predictions of Any Classifier*. Available at: https://arxiv.org/abs/1602.04938.
20. Selvaraju, R.R. *et al.* (2017) "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," in *2017 IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626. Available at: https://doi.org/10.1109/ICCV.2017.74.
21. Wen, B. *et al.* (2025) "Towards a transparent and interpretable AI model for medical image classifications," *Cognitive Neurodynamics*, 19(1). Available at: https://doi.org/10.1007/s11571-025-10343-w.
22. Zeineldin, R. (2023) *Deep multimodality image-guided system for assisting neurosurgery*. Doctoral dissertation. Available at: https://doi.org/10.5445/IR/1000155782.
23. Zeineldin, R.A. *et al.* (2022) "Explainability of deep neural networks for MRI analysis of brain tumors," *International Journal of Computer Assisted Radiology and Surgery*, pp. 1–11.
24. Çiçek, Ö. *et al.* (2016) "3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation," in S. Ourselin et al. (eds.) *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2016*. Cham: Springer International Publishing, pp. 424–432.

Appendix A - Appendices and Supplementary Materials

A.1 Appendix A: Project Administration Forms

A.1.1 A.1 Initial Project Proposal

Initial Project Proposal Form (CN6000)

Unveiling Model Rationales: An Explainable AI Framework for Interpreting Black-Box Tumor Predictions for Convolutional Neural Networks

u2536809

Proposed Aim

The primary aim is to develop and evaluate an Explainable AI (XAI) framework that transforms opaque Deep Learning (DL) models into transparent and trustworthy decision-support systems for automated brain tumor diagnosis and screening.

Proposed Objectives

1. To establish and analyse the distinct mechanisms for localizing relevant images features and quantifying features contributions to the final CNN model prediction.
2. To investigate the fundamental trade-off between model accuracy and model transparency in high-stakes images, establishing a balanced rationale for the integration of the chosen XAI suite.
3. To quantitatively evaluate the precision of the visual XAI output.
4. To conduct quantitative analysis (Robustness testing) of the features contributions using XAI methodologies.
5. To implement and train the proposed CNN model using available MRI images Dataset and integrate it with the post-hoc XAI techniques to create unified Explainable hybrid model.
6. To utilize the interpretability insights provided by XAI to prune CNN model to maintain high accuracy on unseen data.
7. To reflect in the limitations of the implemented hybrid model and propose future research directions.

1

Draft of Rationale

The integration of Explainable AI into deep learning models for MRI-based brain tumor diagnosis is essential for clinical trust, technical reliability, and regulatory compliance. Models like CNN can predict brain tumors by scanning MRI images, but their "Black Box" nature limits safe medical use. XAI addresses this by making model decisions explainable and interpretable, showing which features play a vital role for tumor detection. AI systems ensure alignment with real tumor characteristics. This process not only builds clinician confidence and helps decision-making but also helps detect bias and improve model efficiency, enabling responsible and trustworthy deployment of AI in neuro-oncology.

Facilities Required

Dataset

- Brain tumor MRI Images that are publicly available.

Computational Requirement

- Local computer or university facilities.
- Cloud-based platforms such as Google Colab or Kaggle.

Software Requirements

- Deep learning Framework (Pytorch, Tenserflow, etc).
- Explainability libraries: SHAP, LIME, Grad-CAM implementations.

Supervisor

Maimoona Sharif

2

A.1.2 A.2 Final Project Proposal

Final Project Proposal Form (CN6000)

"Illuminating the Black Box: An Explainable AI Framework for Brain Tumor Segmentation Using Volumetric Data Interpretation in 3D CNNs"

Yogi Amitkumar Patel u2536809

Proposed Aim

The primary goal is to create and assess a new Explainable AI (XAI) framework for volumetric brain tumour segmentation by interpreting 3D Convolutional Neural Network models. By creating 3D voxel-based density maps, this framework will turn opaque 3D models into transparent systems. A significant innovation is the visualisation of these explanations in an immersive Virtual Reality (VR) environment, giving doctors a user-friendly and interactive decision-support tool.

Proposed objectives

1. To identify and evaluate the various processes for 3D, **voxel-based XAI** in order to quantify their contribution to 3D CNN segmentation predictions and localise pertinent features from multimodal 3D MRI volumes.
2. To implement and train a 3D CNN segmentation model using **PyTorch** and **MONAI** on the available dataset subset.
3. To create a **VR visualisation pipeline** that uses 3D Slicer and the SlicerVR extension to export the original 3D MRI, the 3D segmentation mask, and the 3D XAI saliency volume into NIfTI format (.nii.gz) for interactive, immersive analysis.
4. To integrate the trained 3D CNN with the established 3D XAI techniques to produce a single explainable hybrid model that can produce 3D prominent volumes.
5. to **quantitatively** examine the basic trade-off between the 3D model's spatial transparency and segmentation accuracy. This includes confirming the 3D XAI feature attributions' quantitative robustness.
6. to compare the visual saliency maps with accepted clinical information in order to qualitatively assess the accuracy of the 3D XAI output.

1

7. Using XAI's 3D interpretability insights, **prune** the 3D CNN model with the goal of reducing parameters while maintaining good segmentation accuracy on unknown 3D medical data.

Rationale

The **"Black Box"** nature of 3D CNNs, such as the **3D U-Net**, restricts clinical trust and safe adoption, despite its great accuracy in segmenting volumetric brain tumours. Standard 2D XAI heatmaps are not adequate for volumetric analysis, but clinicians need explanations that align with their 3D workflow.

By creating a 3D XAI framework that produces 3D saliency volumes (such as **3D Grad-CAM**) to explain the 3D CNN's voxel-level decisions, this study fills this gap. The main innovation is the development of a pipeline that uses 3D Slicer with SlicerVR to visualise these 3D explanations in an immersive virtual reality (VR) environment.

By enabling physicians to intuitively understand the model's logic, our 3D XAI-VR approach boosts confidence, facilitates decision-making, and offers the insights required for XAI-driven model pruning.

Facilities Required

Dataset

- Primary Dataset: BraTS 2021, a large dataset of multimodal 3D MRI scans (T1, T1ce, T2, FLAIR) in NIfTI and DICOM format.

Computational Requirement

- GPU: A local or cloud-based GPU with 16GB+ VRAM is required for training 3D CNN models on the full dataset.

Software Requirements

- Deep Learning Framework: PyTorch and MONAI
- Explainability Libraries: SHAP, LIME, and TorchCAM (or other 3D Grad-CAM implementations).
- VR Visualization: 3D Slicer with the SlicerVR extension
- 3D Rendering Utilities: ITK-SNAP

2

Hardware Requirements

- VR Headset: A Meta Quest 3

Supervisor

Maimoona Sharif

3

A.1.3 A.3 Internal Ethical Approval Process

11/26/25, 11:55 AM CN6000 Internal Ethical Approval Process 2025

ETHICS

CN6000 Internal Ethical Approval Process 2025

Only complete this form if you have already received approval for the Proposal. If you complete this form before getting the final approval of your proposal, this form will be deleted, and you will be required to do it again!

Do you need internal approval?

This section aims to check if you need internal approval in the first place. If you are not applying field research, then you may not need internal or external approval.

1. Have you received the final Proposal Approval? That is the proposal has been reviewed by the 'second supervisor' and it has been approved? *
   - (selected) Yes
   - No
   - Maybe

2. My study will not involve collecting any data from humans. It is either just based on published datasets that I did not create or there are no datasets at all part of my study. *
   - (selected) True
   - False

3. Please state your topic title: *

   Illuminating the black box: an explainable AI framework for brain tumor segmentation using volumetric data interpretation in 3D CNNs.

4. Please state your supervisor's name (Spell it correctly). *

   Mimoona Maqsood SHARIF

5. If you answered **Yes** to question 1 and **True** to Question 2 - You are not required to proceed. Just click NEXT then Submit without the need to answer the remaining questions. Otherwise, please proceed to section 2. *
   - (selected) True
   - False

This section aims to see if we can approve your proposal internally.

Please read the questions carefully. Feel free to discuss these questions with your supervisor. If you find out that you have to apply for approval by the Research Ethics Committee, please discuss with your supervisor if there is a way to modify your dissertation in a way that does not take away from the value of your work.

6. I intend to plan and conduct my study as a pilot test of the research only.
   - Yes
   - No
   - Maybe

7. I intend to conduct my field research as pilot test on UEL students or staff only.
   - Yes
   - No
   - Maybe

8. I will debrief participants before the study and mention that their contribution is entirely voluntary.
   - Yes
   - No
   - Maybe

9. I will not be collecting any personal data such as names, addresses, telephone numbers, emails...etc.
   - Yes
   - No
   - Maybe

10. I will not be asking any sensitive questions that can cause distress - such as mental health related or emotional questions.
    - Yes
    - No
    - Maybe

11. I will be getting approval of the format and content of my questions from my supervisor before I proceed with my field research.
    - Yes
    - No
    - Maybe

12. I will have a 'consent' form for participants to be part of their study and be informed of their rights.
    - Yes
    - No
    - Maybe

13. I will store the data I collect on UEL OneDrive. No other copy of it will be stored elsewhere.
    - Yes
    - No
    - Maybe

14. I will not use the data for any other intended purpose and delete it at the end of the term.

11/26/25, 11:55 AM CN6000 Internal Ethical Approval Process 2025

- Yes
- No
- Maybe

15. If you have answered No or Maybe to any of the questions above, you cannot proceed with internal Ethical approval. I would advise you to review those particular questions with your supervisor before proceeding. If you both agree that you want to keep these answers, then please indicate below that you are happy to proceed.

- All my answers are Yes. I have completed the internal approval and promise to abide by it.
- Not all my answers are yes. I have spoken to my supervisor and we both agreed that I should apply for form B Ethical Approval process.

16. I understand that violating any of the terms listed above could result in an automatic fail and possibly academic misconduct proceedings.

- I confirm

Microsoft 365

https://forms.office.com/Pages/ResponseDetailPage.aspx?id=aaKUx5sud0etRi0Z2BGW1qS2Ljw3259NmgdTaKKknKJUMU8yVThTWVRaV0ZQWV… 5/5

A.2 Appendix B: Model Configuration and Hyperparameters

This section details the complete configuration YAML used for training the SegResNet model on the BraTS 2023 dataset, ensuring reproducibility of the hyperparameters, optimiser settings, and loss functions.

Listing: Full Training Configuration (`full_training_segresnet.yaml`)

```yaml
project:
  name: "BraTS-Dissertation-Full-SegResNet"
  seed: 42

data:
  data_dir: "data/BraTS2023-Training"       # full training dataset
  train_split: 1000
  val_split: 125                             # remaining ~125 go to test
  roi_size: [160, 160, 160]                  # 160³ - larger spatial context
  batch_size: 1                              # ~38 GB on single GPU with AMP
  num_workers: 8                             # keeps data pipeline saturated
  num_samples: 4                             # RandCropByPosNegLabeld samples

model:
  architecture: "SegResNet"
  spatial_dims: 3
  in_channels: 4
  out_channels: 3
  init_filters: 32
  dropout_prob: 0.1
```

```yaml
training:
  epochs: 35                          # ~50 epochs typical for full BraTS
dataset
  learning_rate: 0.00005
  weight_decay: 0.00001
  scheduler: "cosine"
  loss: "DiceFocalLoss"
  loss_params:
    sigmoid: true
    gamma: 2.0                        # Focal factor for hard examples (ET)
```

A.3 Appendix C: Volumetric Data Transformations

This snippet demonstrates the data preprocessing pipeline implemented in MONAI, including the custom `ConvertToMultiChannelBraTS2023d` transform that maps raw BraTS labels into the required clinical sub-regions.

Listing: Data Transforms Pipeline (`transforms.py`)

```python
"""
transforms.py - BraTS-specific transforms and preprocessing pipelines.
"""

import torch
from monai.transforms import (
    Compose,
    LoadImaged,
    EnsureChannelFirstd,
    EnsureTyped,
    CropForegroundd,
    NormalizeIntensityd,
    SpatialPadd,
    RandCropByPosNegLabeld,
    RandFlipd,
    RandGaussianNoised,
    RandRotate90d,
    RandScaleIntensityd,
    RandShiftIntensityd,
    MapTransform,
)


class ConvertToMultiChannelBraTS2023d(MapTransform):
    """
    Groups raw BraTS labels (1, 2, 3) into clinical sub-regions:
        - Channel 0: Whole Tumour (WT) - labels 1, 2, 3
        - Channel 1: Tumour Core (TC) - labels 1, 3
        - Channel 2: Enhancing Tumour (ET) - label 3
    """

    def __call__(self, data):
```

```
        d = dict(data)
        for key in self.keys:
            result = []
            result.append(
                torch.logical_or(
                    torch.logical_or(d[key] == 1, d[key] == 2), d[key] == 3
                )
            )
            result.append(torch.logical_or(d[key] == 1, d[key] == 3))
            result.append(d[key] == 3)
            d[key] = torch.cat(result, dim=0).float()
        return d


def get_train_transforms(cfg: dict) -> Compose:
    """Build the training transform pipeline from config."""
    roi_size = list(cfg["data"]["roi_size"])
    num_samples = cfg["data"]["num_samples"]

    return Compose([
        LoadImaged(keys=["image", "label"]),
        EnsureChannelFirstd(keys=["image", "label"]),
        EnsureTyped(keys=["image", "label"]),
        CropForegroundd(keys=["image", "label"], source_key="image"),
        NormalizeIntensityd(keys="image", nonzero=True, channel_wise=True),
        ConvertToMultiChannelBraTS2023d(keys="label"),
        SpatialPadd(keys=["image", "label"], spatial_size=roi_size),
        RandCropByPosNegLabeld(
            keys=["image", "label"],
            label_key="label",
            spatial_size=roi_size,
            pos=1,
            neg=1,
            num_samples=num_samples,
        ),
        RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=0),
        RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=1),
        RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=2),
        RandRotate90d(keys=["image", "label"], prob=0.5, max_k=3),
        RandScaleIntensityd(keys="image", factors=0.1, prob=0.5),
        RandShiftIntensityd(keys="image", offsets=0.1, prob=0.5),
        RandGaussianNoised(keys=["image"], prob=0.1, mean=0.0, std=0.1),
    ])


def get_val_transforms(cfg: dict) -> Compose:
    """Build the validation/test transform pipeline from config."""
    roi_size = list(cfg["data"]["roi_size"])

    return Compose([
```

```
        LoadImaged(keys=["image", "label"]),
        EnsureChannelFirstd(keys=["image", "label"]),
        EnsureTyped(keys=["image", "label"]),
        CropForegroundd(keys=["image", "label"], source_key="image"),
        NormalizeIntensityd(keys="image", nonzero=True, channel_wise=True),
        ConvertToMultiChannelBraTS2023d(keys="label"),
        SpatialPadd(keys=["image", "label"], spatial_size=roi_size),
    ])
```

A.4 Appendix D: Training Framework

This section details the custom training loop implemented in PyTorch and MONAI, featuring Automatic Mixed Precision (AMP) for efficiency, Weights & Biases (W&B) integration for metric tracking, and robust checkpointing capabilities.

Listing: Training Loop Implementation (`trainer.py`)

```
"""
trainer.py - Training loop with validation, checkpointing, and W&B logging.

Supports:
  - Automatic Mixed Precision (AMP) for ~2× speedup and ~50% less VRAM
  - Best-model checkpointing (saves when val Dice improves)
  - Last-checkpoint saving (saves every epoch for crash recovery)
  - Resume from last checkpoint (restore model, optimiser, scheduler, epoch)
"""

import os
import torch
import wandb
from tqdm import tqdm

import monai
from monai.transforms import AsDiscrete
from monai.metrics import DiceMetric

from src.config import save_config


class Trainer:
    """Handles the full training lifecycle: train, validate, checkpoint."""

    def __init__(self, model, loaders, loss_fn, optimiser, scheduler, cfg, device):
        self.model = model
        self.loaders = loaders
        self.loss_fn = loss_fn
        self.optimiser = optimiser
        self.scheduler = scheduler
        self.cfg = cfg
        self.device = device
```

```
        self.epochs = cfg["training"]["epochs"]
        self.roi_size = cfg["data"]["roi_size"]
        self.run_dir = cfg["_run_dir"]

        self.post_trans = AsDiscrete(threshold=0.5)
        self.dice_metric = DiceMetric(include_background=True,
reduction="mean_batch")
        self.dice_regions = ["WT", "TC", "ET"]
        self.best_metric = -1
        self.start_epoch = 0
        self.best_model_path = os.path.join(self.run_dir, "best_model.pth")
        self.last_checkpoint_path = os.path.join(self.run_dir,
"last_checkpoint.pth")

        # Automatic Mixed Precision
        self.scaler = torch.amp.GradScaler("cuda")
        self.use_amp = device.type == "cuda"

    def save_last_checkpoint(self, epoch: int):
        """Save a full training snapshot for resume capability."""
        # Unwrap DataParallel so checkpoints are portable
        state_dict = (
            self.model.module.state_dict()
            if hasattr(self.model, "module")
            else self.model.state_dict()
        )
        checkpoint = {
            "epoch": epoch,
            "model_state_dict": state_dict,
            "optimiser_state_dict": self.optimiser.state_dict(),
            "scheduler_state_dict": self.scheduler.state_dict(),
            "scaler_state_dict": self.scaler.state_dict(),
            "best_metric": self.best_metric,
        }
        torch.save(checkpoint, self.last_checkpoint_path)

    def resume(self):
        """Load training state from last checkpoint. Call before fit().

        Returns True if checkpoint was loaded, False otherwise.
        """
        if not os.path.exists(self.last_checkpoint_path):
            print("[Resume] No checkpoint found - training from scratch.")
            return False

        print(f"[Resume] Loading checkpoint: {self.last_checkpoint_path}")
        checkpoint = torch.load(
            self.last_checkpoint_path, map_location=self.device, weights_only=True
        )
```

```
        # Restore model weights (handle DataParallel)
        if hasattr(self.model, "module"):
            self.model.module.load_state_dict(checkpoint["model_state_dict"])
        else:
            self.model.load_state_dict(checkpoint["model_state_dict"])

        self.optimiser.load_state_dict(checkpoint["optimiser_state_dict"])
        self.scheduler.load_state_dict(checkpoint["scheduler_state_dict"])
        if "scaler_state_dict" in checkpoint:
            self.scaler.load_state_dict(checkpoint["scaler_state_dict"])
        self.best_metric = checkpoint["best_metric"]
        self.start_epoch = checkpoint["epoch"] + 1  # resume from next epoch

        print(f"[Resume] Resuming from epoch {self.start_epoch + 1}/{self.epochs}")
        print(f"[Resume] Best Dice so far: {self.best_metric:.4f}")
        return True

    def fit(self):
        """Run the full training loop."""
        # Save config alongside checkpoints
        save_config(self.cfg, self.run_dir)

        remaining = self.epochs - self.start_epoch
        total_pbar = tqdm(
            total=remaining,
            desc="Training",
            position=0,
            initial=0,
        )

        for epoch in range(self.start_epoch, self.epochs):
            train_loss = self._train_epoch(epoch)
            val_results = self._validate_epoch(epoch)
            val_dice = val_results["mean"]

            self.scheduler.step()

            # Log everything in one call so charts share the same x-axis
            wandb.log({
                "epoch": epoch + 1,
                "train_loss": train_loss,
                "val_dice": val_dice,
                "val_dice_WT": val_results["WT"],
                "val_dice_TC": val_results["TC"],
                "val_dice_ET": val_results["ET"],
                "lr": self.optimiser.param_groups[0]["lr"],
            })

            # Checkpoint best model
            if val_dice > self.best_metric:
```

```
                self.best_metric = val_dice
                # Unwrap DataParallel so checkpoints are portable
                state_dict = self.model.module.state_dict() \
                    if hasattr(self.model, "module") else self.model.state_dict()
                torch.save(state_dict, self.best_model_path)
                tqdm.write(f"  → New Best Dice: {self.best_metric:.4f}")

            # Save last checkpoint every epoch (for crash recovery)
            self.save_last_checkpoint(epoch)

            total_pbar.update(1)

        total_pbar.close()

        print(f"\nTraining complete. Best Dice: {self.best_metric:.4f}")
        print(f"Best model saved to: {self.best_model_path}")

    def _train_epoch(self, epoch: int) -> float:
        """Run one training epoch. Returns average loss."""
        self.model.train()
        epoch_loss = 0
        loader = self.loaders["train"]
        step_pbar = tqdm(
            loader, desc=f"↳ Epoch {epoch + 1}", position=1, leave=False
        )

        for batch_data in step_pbar:
            inputs = batch_data["image"].to(self.device)
            labels = batch_data["label"].to(self.device)

            self.optimiser.zero_grad()

            # Forward pass with AMP
            with torch.amp.autocast("cuda", enabled=self.use_amp):
                outputs = self.model(inputs)
                loss = self.loss_fn(outputs, labels)

            # Backward pass with gradient scaling
            self.scaler.scale(loss).backward()
            self.scaler.step(self.optimiser)
            self.scaler.update()

            epoch_loss += loss.item()
            step_pbar.set_postfix({"loss": f"{loss.item():.4f}"})
            wandb.log({"batch_loss": loss.item()})

        return epoch_loss / len(loader)

    def _validate_epoch(self, epoch: int) -> dict:
        """Run validation with sliding window inference. Returns dict of Dice
```

```
scores."""
        self.model.eval()
        with torch.no_grad(), torch.amp.autocast("cuda", enabled=self.use_amp):
            for val_data in self.loaders["val"]:
                v_in = val_data["image"].to(self.device)
                v_lab = val_data["label"].to(self.device)
                v_out = monai.inferers.sliding_window_inference(
                    v_in, self.roi_size, 4, self.model
                )
                v_out = [self.post_trans(torch.sigmoid(i)) for i in v_out]
                self.dice_metric(y_pred=v_out, y=v_lab)

            results = self.dice_metric.aggregate()
            self.dice_metric.reset()

        return {
            region: results[i].item()
            for i, region in enumerate(self.dice_regions)
        } | {"mean": results.mean().item()}
```

A.5 Appendix E: Explainable AI (XAI) Suite

The following sections document the complete source code for the post-hoc Explainable AI framework, including gradient-based methods, perturbation methods, uncertainty quantification, and quantitative validation metrics.

A.5.1 E.1 3D Gradient-weighted Class Activation Mapping (Grad-CAM)

Listing: 3D Grad-CAM Implementation (`grad_cam.py`)

```
"""
grad_cam.py - 3D Grad-CAM for volumetric segmentation models.

Implements Gradient-weighted Class Activation Mapping (Grad-CAM) for 3D CNNs.
Produces a class-discriminative saliency volume by weighting feature-map
activations at a chosen convolutional layer with gradient-derived importance
weights.

Reference:
    Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks
    via Gradient-based Localisation", IJCV 2020.

Usage:
    from src.xai.grad_cam import GradCAM3D

    gcam = GradCAM3D(model, target_layer=model.down_layers[-1])
    heatmap = gcam.generate(input_tensor, target_class=2)  # ET channel
    gcam.remove_hooks()
"""

import torch
import torch.nn.functional as F
```

```python
import numpy as np
from typing import Optional


class GradCAM3D:
    """
    3D Grad-CAM for volumetric segmentation models.

    Hooks into a target convolutional layer, captures activations during the
    forward pass and gradients during the backward pass, then computes a
    class-discriminative 3D heatmap.

    Args:
        model:        A trained 3D segmentation model (e.g. SegResNet).
        target_layer: The nn.Module (conv layer/block) to hook into.
                      Typically the last encoder block (bottleneck).
    """

    def __init__(self, model: torch.nn.Module, target_layer: torch.nn.Module):
        self.model = model
        self.target_layer = target_layer

        # Storage for hooked tensors
        self._activations: Optional[torch.Tensor] = None
        self._gradients: Optional[torch.Tensor] = None

        # Register hooks
        self._forward_hook =
target_layer.register_forward_hook(self._save_activation)
        self._backward_hook =
target_layer.register_full_backward_hook(self._save_gradient)

    # ── Hook callbacks ──────────────────────────────────────────

    def _save_activation(self, module, input, output):
        """Forward hook: store the layer's output activations."""
        self._activations = output.detach()

    def _save_gradient(self, module, grad_input, grad_output):
        """Backward hook: store the gradients flowing into this layer."""
        self._gradients = grad_output[0].detach()

    # ── Core Grad-CAM computation ───────────────────────────────

    def generate(
        self,
        input_tensor: torch.Tensor,
        target_class: int,
        upsample: bool = True,
    ) -> np.ndarray:
```

```
        """
        Generate a 3D Grad-CAM heatmap for a given class channel.

        Args:
            input_tensor:  Model input, shape (1, C, D, H, W).
            target_class:  Output channel index (0=WT, 1=TC, 2=ET).
            upsample:      If True, trilinearly upsample the heatmap to the
                           input spatial resolution. Otherwise return at
                           feature-map resolution.

        Returns:
            Numpy array (D, H, W) with values in [0, 1].
        """
        self.model.eval()

        # ── 1. Forward pass (activations are captured by the hook) ──────
        output = self.model(input_tensor)

        # ── 2. Compute scalar score y^c ─────────────────────────────────
        # For segmentation: spatially average the pre-sigmoid logits for the
        # target class channel to obtain a single scalar.
        score = output[0, target_class].mean()

        # ── 3. Backward pass (gradients are captured by the hook) ───────
        self.model.zero_grad()
        score.backward(retain_graph=False)

        # ── 4. Importance weights: global-average-pool the gradients ────
        # _gradients shape: (1, K, D', H', W')
        # → alpha shape:    (1, K, 1, 1, 1)  for broadcasting
        alpha = self._gradients.mean(dim=[2, 3, 4], keepdim=True)

        # ── 5. Weighted combination of feature maps ─────────────────────
        # _activations shape: (1, K, D', H', W')
        cam = (alpha * self._activations).sum(dim=1, keepdim=True)  # (1, 1, D',
H', W')

        # ── 6. ReLU - keep only positive contributions ──────────────────
        cam = F.relu(cam)

        # ── 7. Normalise to [0, 1] ──────────────────────────────────────
        cam_min = cam.min()
        cam_max = cam.max()
        cam = (cam - cam_min) / (cam_max - cam_min + 1e-8)

        # ── 8. Upsample to input spatial resolution ─────────────────────
        if upsample:
            spatial_size = input_tensor.shape[2:]  # (D, H, W)
            cam = F.interpolate(
                cam, size=spatial_size, mode="trilinear", align_corners=False
```

```
            )

        # Return as numpy: squeeze batch and channel dims → (D, H, W)
        return cam.squeeze().cpu().numpy()

    # ── Cleanup ──────────────────────────────────────────────────────────

    def remove_hooks(self):
        """Remove all registered hooks. Call this when done."""
        self._forward_hook.remove()
        self._backward_hook.remove()

    def __del__(self):
        """Safety net: remove hooks on garbage collection."""
        try:
            self.remove_hooks()
        except Exception:
            pass
```

A.5.2 E.2 3D Guided Backpropagation

Listing: 3D Guided Backpropagation Implementation (`guided_backprop.py`)

```
"""
guided_backprop.py - 3D Guided Backpropagation for volumetric segmentation models.

Implements Guided Backpropagation (GBP), which modifies the standard gradient
backpropagation by additionally gating negative gradients at every ReLU layer.
This produces a full-resolution, pixel-level saliency map that highlights
fine-grained features the model uses.

Unlike Grad-CAM (which is coarse but class-discriminative), GBP produces
sharp, high-resolution maps but is NOT class-discriminative on its own.
Combining GBP with Grad-CAM yields Guided Grad-CAM (sharp + class-specific).

Reference:
    Springenberg et al., "Striving for Simplicity: The All Convolutional Net",
    ICLR Workshop 2015.

Usage:
    from src.xai.guided_backprop import GuidedBackprop3D

    gbp = GuidedBackprop3D(model)
    saliency = gbp.generate(input_tensor, target_class=0)  # WT channel
    gbp.restore_relus()
"""

import torch
import torch.nn.functional as F
import numpy as np
from typing import Optional, List, Tuple
```

```python
class GuidedBackprop3D:
    """
    3D Guided Backpropagation for volumetric segmentation models.

    Replaces all ReLU activations in the model with "Guided ReLUs" that
    gate both negative activations (standard ReLU) and negative gradients
    during the backward pass. This produces a full-resolution saliency map.

    Args:
        model: A trained 3D segmentation model (e.g. SegResNet).
    """

    def __init__(self, model: torch.nn.Module):
        self.model = model
        self._hooks: List[torch.utils.hooks.RemovableHook] = []
        self._original_relus: List[Tuple[torch.nn.Module, str, torch.nn.Module]] =
[]

        # Replace all ReLU layers with guided versions
        self._override_relus(model)

    # ── ReLU Override ──────────────────────────────────────────────────

    def _guided_relu_hook(self, module, grad_input, grad_output):
        """
        Backward hook for Guided Backpropagation.

        Gates the gradient: only pass through gradients that are positive
        AND where the forward activation was also positive.
        The forward ReLU already zeroes negative activations, so this hook
        additionally zeroes negative incoming gradients.
        """
        # grad_output[0] is the gradient flowing backward through this ReLU
        # Clamp: only keep positive gradients
        return (F.relu(grad_output[0]),)

    def _override_relus(self, module: torch.nn.Module):
        """
        Recursively find all ReLU/PReLU layers and register backward hooks
        that implement the guided backprop gradient gating.

        Also disables inplace=True on ReLU layers to prevent conflicts
        when both GBP and Grad-CAM hooks are active on the same model
        (even though we isolate them per-patient, inplace ops can still
        cause issues with autograd).

        For MONAI SegResNet, ReLU activations are inside residual blocks
        as nn.ReLU or act layers.
```

```
        """
        for name, child in module.named_children():
            if isinstance(child, (torch.nn.ReLU, torch.nn.PReLU,
torch.nn.LeakyReLU)):
                # Disable inplace to avoid potential autograd conflicts
                if hasattr(child, 'inplace') and child.inplace:
                    self._original_relus.append((child, 'inplace', True))
                    child.inplace = False
                # Register backward hook on this ReLU
                hook = child.register_full_backward_hook(self._guided_relu_hook)
                self._hooks.append(hook)
            else:
                # Recurse into sub-modules
                self._override_relus(child)

    # ── Core GBP Computation ────────────────────────────────────────────

    def generate(
        self,
        input_tensor: torch.Tensor,
        target_class: int,
    ) -> np.ndarray:
        """
        Generate a full-resolution Guided Backpropagation saliency map.

        Args:
            input_tensor: Model input, shape (1, C, D, H, W).
            target_class: Output channel index (0=WT, 1=TC, 2=ET).

        Returns:
            Numpy array (D, H, W) with values in [0, 1].
            Full input-space resolution saliency map.
        """
        self.model.eval()

        # Ensure input requires gradient for backprop to reach it
        input_tensor = input_tensor.detach().clone()
        input_tensor.requires_grad_(True)

        # ── 1. Forward pass ─────────────────────────────────────────────
        output = self.model(input_tensor)

        # ── 2. Compute scalar score (same as Grad-CAM) ──────────────────
        score = output[0, target_class].mean()

        # ── 3. Backward pass with guided ReLU hooks active ──────────────
        self.model.zero_grad()
        if input_tensor.grad is not None:
            input_tensor.grad.zero_()
        score.backward(retain_graph=False)
```

```
        # ── 4. Extract the input gradient ──────────────────────────────
        # The gradient at the input tells us how much each input voxel
        # contributed to the target class score.
        grad = input_tensor.grad.detach()  # (1, C, D, H, W)

        # ── 5. Aggregate across input channels ─────────────────────────
        # Take the max absolute gradient across all 4 MRI modalities
        # to get a single saliency map showing the most important voxels
        saliency = grad[0].abs().max(dim=0).values  # (D, H, W)

        # ── 6. Normalise to [0, 1] ─────────────────────────────────────
        s_min = saliency.min()
        s_max = saliency.max()
        saliency = (saliency - s_min) / (s_max - s_min + 1e-8)

        return saliency.cpu().numpy()

    # ── Cleanup ────────────────────────────────────────────────────────

    def restore_relus(self):
        """Remove all registered backward hooks and restore inplace state."""
        for hook in self._hooks:
            hook.remove()
        self._hooks.clear()
        # Restore original inplace=True on ReLUs
        for module, attr, original_value in self._original_relus:
            setattr(module, attr, original_value)
        self._original_relus.clear()

    def __del__(self):
        """Safety net: remove hooks on garbage collection."""
        try:
            self.restore_relus()
        except Exception:
            pass
```

A.5.3 E.3 Input × Gradient (LRP Proxy)

Listing: Input × Gradient Implementation (`lrp.py`)

```
"""
lrp.py - Input × Gradient attribution (LRP proxy) for volumetric models.

Implements the Input × Gradient method, a first-order Taylor decomposition
that serves as a tractable proxy for epsilon-LRP in architectures where
true layer-wise propagation is infeasible (e.g., SegResNet with GroupNorm
and residual skip connections).

Unlike Guided Backpropagation, which filters gradients purely to find
"what could change the output", Input × Gradient distributes the actual
```

```python
prediction score back to the input voxels to answer "what explicitly
contributed to the output".
"""

import torch
import numpy as np


class LRP3D:
    """
    3D Layer-wise Relevance Propagation (LRP) via Input x Gradient proxy.

    Computes relevance maps by multiplying the raw input image with the
    gradient of the target class score with respect to the input image.
    This provides a mathematically sound approximation of epsilon-LRP
    for networks primarily using ReLU activations.

    Args:
        model: A trained 3D segmentation model (e.g. SegResNet).
    """

    def __init__(self, model: torch.nn.Module):
        self.model = model

    def generate(
        self,
        input_tensor: torch.Tensor,
        target_class: int,
    ) -> np.ndarray:
        """
        Generate a full-resolution LRP relevance map.

        Args:
            input_tensor: Model input, shape (1, C, D, H, W).
            target_class: Output channel index (0=WT, 1=TC, 2=ET).

        Returns:
            Numpy array (D, H, W) with values in [0, 1].
            Full input-space resolution relevance map.
        """
        self.model.eval()

        # Ensure input requires gradient for backprop to reach it
        input_tensor = input_tensor.detach().clone()
        input_tensor.requires_grad_(True)

        # ── 1. Forward pass ─────────────────────────────────────────────
        output = self.model(input_tensor)

        # ── 2. Compute scalar score ─────────────────────────────────────
```

```
        score = output[0, target_class].mean()

        # ── 3. Backward pass ──────────────────────────────────────
        self.model.zero_grad()
        if input_tensor.grad is not None:
            input_tensor.grad.zero_()
        score.backward(retain_graph=False)

        # ── 4. Extract the input gradient ─────────────────────────
        grad = input_tensor.grad.detach()  # (1, C, D, H, W)

        # ── 5. Input x Gradient ───────────────────────────────────
        # Element-wise multiplication of gradient with original input
        relevance = grad * input_tensor.detach()

        # ── 6. Aggregate across input channels ────────────────────
        # Sum relevance across the 4 MRI modalities (T1, T1ce, T2, FLAIR)
        # to get total relevance per spatial voxel.
        relevance = relevance[0].sum(dim=0)  # (D, H, W)

        # We typically care about positive relevance (evidence FOR the class).
        # We clamp negative relevance to 0.
        relevance = torch.clamp(relevance, min=0)

        # ── 7. Normalise to [0, 1] ────────────────────────────────
        r_min = relevance.min()
        r_max = relevance.max()
        if r_max - r_min > 1e-8:
            relevance = (relevance - r_min) / (r_max - r_min)
        else:
            relevance = torch.zeros_like(relevance)

        return relevance.cpu().numpy()
```

A.5.4 E.4 3D Occlusion Sensitivity

Listing: 3D Occlusion Sensitivity Implementation (`occlusion.py`)

```
"""
occlusion.py - 3D Occlusion Sensitivity for volumetric models.

Implements a sliding-window perturbation method to test model reliance
on specific spatial regions. Unlike gradient methods, this physically hides
portions of the input and records the drop in target prediction confidence.
"""

import torch
import torch.nn.functional as F
import numpy as np
from tqdm import tqdm
```

```python
class OcclusionSensitivity3D:
    """
    3D Occlusion Sensitivity (Sliding Window).

    Args:
        model:          A trained 3D segmentation model.
        window_size:    Size of the occluding patch (D, H, W).
        stride:         Step size for the sliding window (D, H, W).
        baseline:       Value to replace occluded pixels with (e.g. 0).
    """

    def __init__(
        self,
        model: torch.nn.Module,
        window_size: tuple = (16, 16, 16),
        stride: tuple = (8, 8, 8),
        baseline: float = 0.0,
    ):
        self.model = model
        self.window_sizes = window_size
        self.strides = stride
        self.baseline = baseline

    @torch.no_grad()
    def generate(
        self,
        input_tensor: torch.Tensor,
        batch_size: int = 16,
    ) -> np.ndarray:
        """
        Generate a 3D occlusion sensitivity relevance map for all 3 classes
simultaneously.

        Args:
            input_tensor: Model input, shape (1, C, D, H, W).
            batch_size:   (Deprecated, unused in sequential to save RAM)

        Returns:
            Numpy array (3, D, H, W) with values in [0, 1]. Size matches input
spatial dims.
        """
        self.model.eval()
        device = input_tensor.device

        # Get spatial dimensions (1, C, D, H, W) -> (D, H, W)
        _, _, D, H, W = input_tensor.shape

        # 1. Base Unoccluded Score for all 3 classes [0, 1, 2]
        base_output = self.model(input_tensor)
```

```
        base_score_0 = base_output[0, 0].mean().item()
        base_score_1 = base_output[0, 1].mean().item()
        base_score_2 = base_output[0, 2].mean().item()

        # 2. Setup sliding window grid
        d_stride, h_stride, w_stride = self.strides
        d_win, h_win, w_win = self.window_sizes

        d_coords = list(range(0, D - d_win + 1, d_stride))
        if d_coords[-1] + d_win < D: d_coords.append(D - d_win)

        h_coords = list(range(0, H - h_win + 1, h_stride))
        if h_coords[-1] + h_win < H: h_coords.append(H - h_win)

        w_coords = list(range(0, W - w_win + 1, w_stride))
        if w_coords[-1] + w_win < W: w_coords.append(W - w_win)

        # Output heatmap corresponding to stride grid centers for 3 classes
        occlusion_map = torch.zeros(
            (3, len(d_coords), len(h_coords), len(w_coords)),
            device=device,
            dtype=torch.float32
        )

        # 3. Process occlusions sequentially to save RAM (no batching)
        # We modify the input_tensor in-place, forward pass, and then restore it.
        total_steps = len(d_coords) * len(h_coords) * len(w_coords)

        with tqdm(total=total_steps, desc="  Sliding", leave=False) as pbar:
            for i_d, d in enumerate(d_coords):
                for i_h, h in enumerate(h_coords):
                    for i_w, w in enumerate(w_coords):

                        # Save the original patch
                        orig_patch = input_tensor[
                            0, :, d : d + d_win, h : h + h_win, w : w + w_win
                        ].clone()

                        # Apply occlusion
                        input_tensor[
                            0, :, d : d + d_win, h : h + h_win, w : w + w_win
                        ] = self.baseline

                        # Forward pass (batch size 1) evaluates all 3 classes at
once

                        occ_output = self.model(input_tensor)
                        occ_0 = occ_output[0, 0].mean().item()
                        occ_1 = occ_output[0, 1].mean().item()
                        occ_2 = occ_output[0, 2].mean().item()
```

```
                        # Record drop
                        occlusion_map[0, i_d, i_h, i_w] = base_score_0 - occ_0
                        occlusion_map[1, i_d, i_h, i_w] = base_score_1 - occ_1
                        occlusion_map[2, i_d, i_h, i_w] = base_score_2 - occ_2

                        # Restore original patch
                        input_tensor[
                            0, :, d : d + d_win, h : h + h_win, w : w + w_win
                        ] = orig_patch

                        pbar.update(1)

        # 4. Upsample strided map to full original input resolution
        # occlusion_map is currently (3, D', H', W'). Add dummy batch for
F.interpolate -> (1, 3, D', H', W')
        occlusion_map = occlusion_map.unsqueeze(0)

        # Upsample via trilinear interpolation
        full_map = F.interpolate(
            occlusion_map,
            size=(D, H, W),
            mode="trilinear",
            align_corners=False
        ).squeeze(0)  # Remove dummy batch dim -> (3, D, H, W)

        # 5. Only care about POSITIVE drops (occlusion hurt performance)
        full_map = torch.clamp(full_map, min=0)

        # 6. Normalize to [0, 1] per channel
        for c in range(3):
            m_min = full_map[c].min()
            m_max = full_map[c].max()
            if m_max - m_min > 1e-8:
                full_map[c] = (full_map[c] - m_min) / (m_max - m_min)
            else:
                full_map[c] = torch.zeros_like(full_map[c])

        return full_map.cpu().numpy()
```

A.5.5 E.5 Monte Carlo (MC) Dropout

Listing: MC Dropout Uncertainty Implementation (`uncertainty.py`)

```
"""
uncertainty.py - MC Dropout for uncertainty quantification in 3D segmentation.

Implements Monte Carlo (MC) Dropout to estimate model uncertainty. By running
multiple forward passes with dropout layers enabled at inference time, we can
compute the mean prediction (a more stable segmentation) and the voxel-wise
variance (a proxy for predictive uncertainty).
```

```
Key insight from notes: No model retraining needed - SegResNet already has
dropout_prob=0.1 in its architecture. We just keep those layers active
during inference.

Reference:
    Gal & Ghahramani, "Dropout as a Bayesian Approximation:
    Representing Model Uncertainty in Deep Learning", ICML 2016.

Usage:
    from src.xai.uncertainty import MCDropout3D

    mcd = MCDropout3D(model, num_iters=20)
    mean_pred, uncertainty = mcd.generate(input_tensor)
    mcd.restore()  # re-disable dropout
"""

import torch
import numpy as np


class MCDropout3D:
    """
    Monte Carlo (MC) Dropout for 3D segmentation uncertainty estimation.

    Enables dropout at inference time and runs N stochastic forward passes
    on the same input to obtain:
        - Mean prediction: a stabilised, ensemble-like segmentation mask
        - Uncertainty map: per-voxel variance across passes

    Args:
        model:      A trained 3D segmentation model with Dropout layers.
        num_iters:  Number of stochastic forward passes (default: 20).
    """

    def __init__(self, model: torch.nn.Module, num_iters: int = 20):
        self.model = model
        self.num_iters = num_iters

    def _enable_dropout(self):
        """
        Enable dropout layers during inference while keeping all other
        layers (BatchNorm, etc.) in eval mode.

        This is the core of MC Dropout: model.eval() disables dropout,
        so we selectively re-enable only Dropout modules.
        """
        self.model.eval()
        for module in self.model.modules():
            if module.__class__.__name__.startswith("Dropout"):
                module.train()
```

```python
def _restore_eval(self):
    """Restore full eval mode (disable dropout again)."""
    self.model.eval()

def generate(self, input_tensor: torch.Tensor):
    """
    Generate mean prediction and uncertainty map using MC Dropout.

    Process:
        1. Enable dropout at inference time
        2. Run N forward passes on the same input
        3. Apply sigmoid → probabilities ∈ [0, 1]
        4. Stack all N outputs
        5. Compute mean (stable prediction) and variance (uncertainty)

    Args:
        input_tensor: Model input, shape (1, C, D, H, W).

    Returns:
        Tuple of:
            mean_pred:       numpy array (3, D, H, W) - mean probabilities
            uncertainty_map: numpy array (3, D, H, W) - per-voxel variance
    """
    self._enable_dropout()

    outputs = []
    with torch.no_grad():
        for _ in range(self.num_iters):
            output = self.model(input_tensor)
            probs = torch.sigmoid(output)
            outputs.append(probs)

    # Stack → (N, 1, 3, D, H, W)
    stacked = torch.stack(outputs)

    # Mean prediction → (1, 3, D, H, W)
    mean_pred = torch.mean(stacked, dim=0)

    # Variance (uncertainty) → (1, 3, D, H, W)
    uncertainty = torch.var(stacked, dim=0)

    # Restore eval mode
    self._restore_eval()

    # Remove batch dim → (3, D, H, W)
    mean_np = mean_pred.squeeze(0).cpu().numpy()
    unc_np = uncertainty.squeeze(0).cpu().numpy()

    return mean_np, unc_np
```

A.5.6 E.6 XAI Evaluation Metrics

Listing: XAI Evaluation Metrics Implementation (`metrics.py`)

```
"""
metrics.py - Quantitative evaluation of XAI saliency maps against ground truth.

Implements metrics that measure how well a model's attention (saliency)
aligns with actual tumour locations, answering:
    "Is the model looking at the right region for the right reasons?"

Metrics:
    - Pointing Game:     Does the peak saliency voxel fall inside the tumour?
    - Saliency Coverage: What fraction of total saliency mass is inside the tumour?
    - Saliency IoU:      Overlap between thresholded saliency and ground truth
mask.

Reference:
    Zhang et al., "Top-Down Neural Attention by Excitation Backprop", IJCV 2018
    (Pointing Game metric)
"""

import numpy as np
from typing import Dict


def pointing_game(saliency: np.ndarray, ground_truth: np.ndarray) -> bool:
    """
    Pointing Game: does the peak saliency voxel fall inside the GT region?

    Args:
        saliency:     3D array (D, H, W) with values in [0, 1].
        ground_truth: 3D binary array (D, H, W), 1 = tumour, 0 = background.

    Returns:
        True if the voxel with maximum saliency is inside the GT mask.
    """
    peak_idx = np.unravel_index(np.argmax(saliency), saliency.shape)
    return bool(ground_truth[peak_idx] == 1)


def msr_accuracy(saliency: np.ndarray, ground_truth: np.ndarray) -> bool:
    """
    Most Salient Region (MSR) Accuracy.

    Similar to Pointing Game, but conceptually used for perturbation methods
    like Occlusion Sensitivity, testing if the single local patch that caused
    the largest confidence drop actually falls inside the anomaly.

    Formula:
        MSR = 1 if G(argmax(S)) == 1 else 0
```

```
    Args:
        saliency:     3D array (D, H, W) of occlusion sensitivity drops.
        ground_truth: 3D binary array (D, H, W), 1 = tumour, 0 = background.

    Returns:
        True if the maximum sensitivity drop voxel is within the tumour.
    """
    return pointing_game(saliency, ground_truth)


def saliency_coverage(saliency: np.ndarray, ground_truth: np.ndarray) -> float:
    """
    Saliency Coverage: fraction of total saliency mass inside the GT region.

    High coverage (→ 1.0) means the model focuses its attention on the
    actual tumour.  Low coverage means the model is distracted by
    irrelevant areas.

    Args:
        saliency:     3D array (D, H, W) with values in [0, 1].
        ground_truth: 3D binary array (D, H, W), 1 = tumour, 0 = background.

    Returns:
        Float in [0, 1].  1.0 = all saliency is inside the tumour.
    """
    total = saliency.sum()
    if total < 1e-8:
        return 0.0
    inside = saliency[ground_truth == 1].sum()
    return float(inside / total)


def saliency_iou(
    saliency: np.ndarray,
    ground_truth: np.ndarray,
    threshold: float = 0.5,
) -> float:
    """
    Saliency IoU: intersection-over-union of thresholded saliency and GT mask.

    Args:
        saliency:     3D array (D, H, W) with values in [0, 1].
        ground_truth: 3D binary array (D, H, W), 1 = tumour, 0 = background.
        threshold:    Saliency values above this are considered "active".

    Returns:
        Float in [0, 1].  1.0 = perfect overlap.
    """
    s_binary = (saliency >= threshold).astype(np.uint8)
```

```
    gt_binary = (ground_truth >= 1).astype(np.uint8)

    intersection = (s_binary & gt_binary).sum()
    union = (s_binary | gt_binary).sum()

    if union == 0:
        return 0.0
    return float(intersection / union)


def weighted_dice(
    saliency: np.ndarray,
    ground_truth: np.ndarray,
) -> float:
    """
    Weighted (Soft) Dice between continuous saliency and binary GT mask.

    Unlike standard Dice (which compares two binary masks), this treats
    the saliency as soft membership values [0, 1] and computes overlap
    with the binary ground truth.  This rewards saliency maps that not
    only cover the tumour but also match its shape.

    Formula:
        Weighted_Dice = 2 * Σ(S · G) / (Σ S + Σ G)

    where S ∈ [0,1] is the normalised saliency and G ∈ {0,1} is the GT.

    Args:
        saliency:     3D array (D, H, W) with values in [0, 1].
        ground_truth: 3D binary array (D, H, W), 1 = tumour, 0 = background.

    Returns:
        Float in [0, 1].  1.0 = perfect soft overlap.
    """
    numerator = 2.0 * (saliency * ground_truth).sum()
    denominator = saliency.sum() + ground_truth.sum()
    if denominator < 1e-8:
        return 0.0
    return float(numerator / denominator)


def evaluate_saliency(
    saliency: np.ndarray,
    ground_truth: np.ndarray,
    threshold: float = 0.5,
) -> Dict[str, float]:
    """
    Compute all saliency-vs-GT metrics in one call.

    Args:
```

```
        saliency:     3D array (D, H, W) with values in [0, 1].
        ground_truth: 3D binary array (D, H, W), 1 = tumour, 0 = background.
        threshold:    Threshold for binarising saliency (for IoU).

    Returns:
        Dict with keys: 'pointing_game', 'msr_accuracy', 'coverage', 'iou',
'weighted_dice'.
    """
    return {
        "pointing_game": float(pointing_game(saliency, ground_truth)),
        "msr_accuracy": float(msr_accuracy(saliency, ground_truth)),
        "coverage": saliency_coverage(saliency, ground_truth),
        "iou": saliency_iou(saliency, ground_truth, threshold),
        "weighted_dice": weighted_dice(saliency, ground_truth),
    }
```